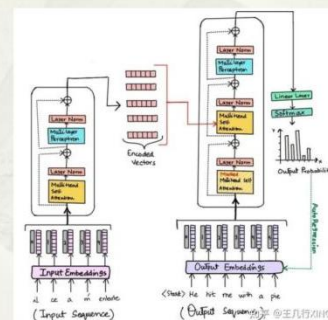
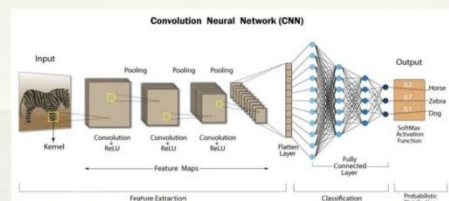
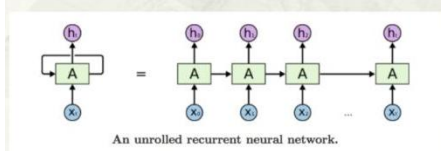
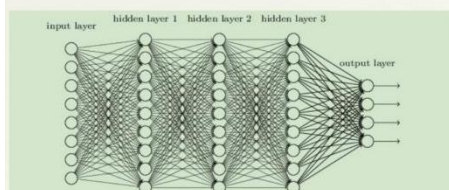
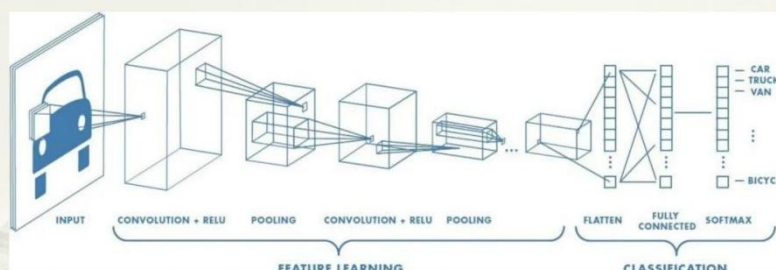


常用深度学习架构



CNN运行流程



CNN模拟感受野机制，通过卷积操作自动提取图像特征

感受野的大小和功能会随着视觉信息处理的层次而变化：小感受野主要负责处理高频信息，如物体的细节；大感受野主要负责处理低频信息，如物体的轮廓。

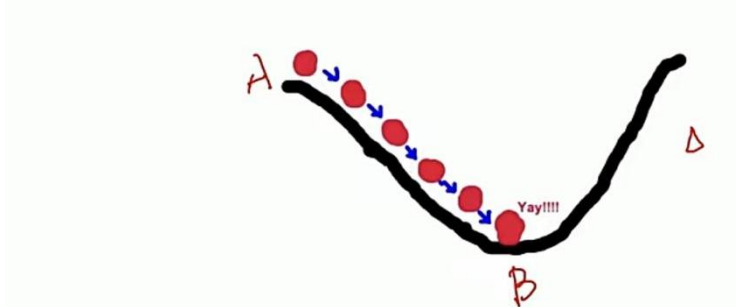
卷积神经网络：将图片 ($w \times h \times c$ ，其中 w 是图片的长； h 是图片的宽； c 为图片的通道数量，若为单通道(灰色图)，则 c 为 1，若为多通道(彩色图)，则 c 为 3) 作为网络的输入，自动提取特征，并且对图片的变形(如平移、比例缩放、倾斜)等具有高度不变性。

卷积神经网络学到的是局部的、平移不变的特征。

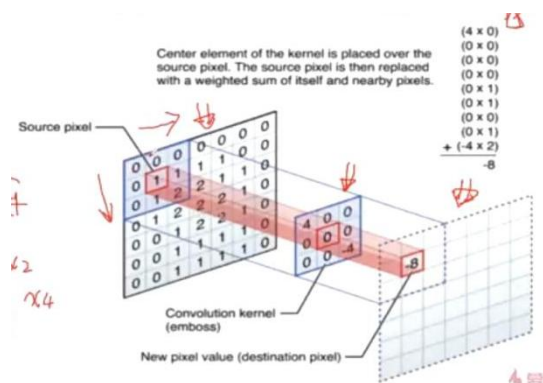
梯度：函数在 A 点无数个变化方向中变化最快的那个方

向。

➤ 记为: ∇f 或者 $\text{grad} f = (\frac{\partial \varphi}{\partial x}, \frac{\partial \varphi}{\partial y}, \frac{\partial \varphi}{\partial z})$



卷积：对图像和滤波矩阵做内积（逐个元素相乘再求和）的操作。每一种卷积对应一种特征。



卷积中的重要参数：

卷积核：最常用为 2D 卷积核 ($k_w \times k_h$)。

常用卷积核： 1×1 , 3×3 , 5×5 。

卷积核是奇数的作用：保护位置信息；padding 时对称。

卷积的两个特性：拥有局部感知机制；权值共享。

1. 局部感知机制

局部感知机制是指网络在处理图像时，每个神经元只对输入图像的一个局部区域（称为感受野）进行响应。

优点：

参数减少：由于每个神经元只处理图像的一部分，因此

所需的参数数量大大减少，这有助于减少模型的复杂度和计算量。

平移不变性：局部感知机制使得模型对图像中的小的平移具有一定的不变性。即，即使图像中的物体发生小的位移，模型仍然能够识别出该物体。

特征提取：通过多层卷积，网络可以学习到图像的局部特征（如边缘、纹理等），这些特征对于图像识别至关重要。

2. 权值共享

权值共享是指在卷积层中，同一个卷积核（或滤波器）在图像的所有位置上使用相同的权重。这意味着，无论卷积核在图像的哪个位置滑动，它都使用相同的权重来计算输出。

优点：

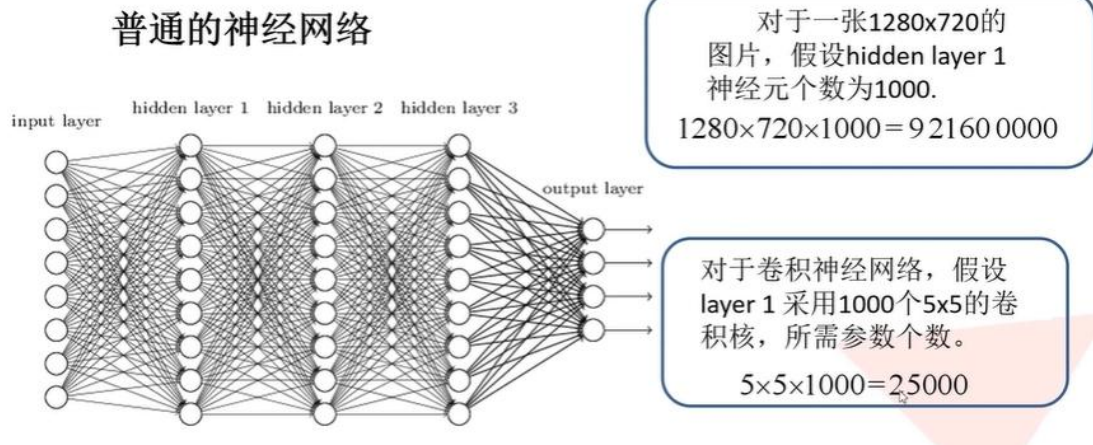
参数减少：权值共享进一步减少了模型的参数数量。由于整个图像上使用相同的权重，因此只需要为每个卷积核存储一组权重，而不是为每个位置存储一组权重。

计算效率：权值共享提高了模型的计算效率。由于权重是固定的，因此在图像的不同位置上进行卷积运算时，可以重复使用相同的计算结果，从而减少了计算量。

特征泛化：权值共享有助于模型学习到图像中的特征的泛化表示。这意味着模型不仅能够识别训练数据中的特征，还能够识别新图像中的相似特征。

总结

局部感知机制使得模型能够捕捉图像的局部特征，而权重共享则减少了模型的参数数量和计算量，同时提高了模型的泛化能力。



卷积运算作用在局部；feature map 使用同一个卷积核运算后得到一种特征；多种特征采用多个卷积核(channel)。

卷积核与感受野：（在步长为 1、无 padding（或 padding 适配）的情况下）

2 个 $3 \times 3 = 1$ 个 5×5 : 2 个 3×3 的卷积核的感受野等于 1 个 5×5 卷积核的感受野。

3 个 $3 \times 3 = 1$ 个 7×7

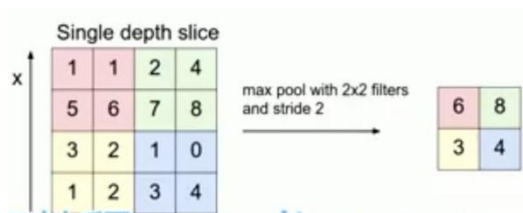
Padding: 通过调整输入的有效尺寸，确保卷积输出的 Feature Map 满足整数倍尺度变化。对尺度相关的任务至关重要——它保证了各层特征图的尺寸对齐。

池化：对输入的特征图进行压缩。

①使特征图变小，简化网络计算复杂度。

②进行特征压缩，提取主要特征。

③增大感受野。



常见的池化策略：最大池化、平均池化、随机池化。

Max Pooling（最大池化）是一种常用的下采样操作。

Max Pooling 层的主要目的是减小特征图的空间尺寸，从而减少模型的参数数量和计算量，同时提取重要特征，增强模型对图像平移的不变性。

BatchNorm（批量归一化）层：核心是通过规范化手段将神经网络每层神经元的输入分布“拉回”到均值为 0、方差为 1 的标准正态分布，从而提升模型训练效率和稳定性。

目的是解决“内部协变量偏移”问题（即训练过程中，每层输入的分布因前层参数更新而持续变化，导致后层训练不稳定），从而加速收敛、提升模型泛化能力。

BatchNorm 层优点：

①减少了参数的人为选择，可以取消 Dropout 和 L2 正则项参数，或者采取更小的 L2 正则项约束参数；

②减少了对学习率的要求；可以不再使用局部响应归一化，BN 本身就是归一化网络；

③破坏原来的数据分布，一定程度上缓解过拟合。

全连接层：连接所有的特征，将输出值送给分类器（如

softmax 分类器)。

全连接层可以将网络的输出变成一个向量。

可以采用卷积代替全连接层。

全连接层因结构特性对输入“尺度”敏感，易引发过拟合；而 Dropout 通过随机丢弃神经元的方式，恰好能抑制这种过拟合，增强模型的泛化能力。两者配合使用，是深度学习中平衡“特征聚合能力”与“泛化稳定性”的经典策略。

Dropout 层：在训练过程中，随机的丢弃一部分输入，此时丢弃部分对应的参数不会更新。

损失层：定义了使用的损失函数(用来评估模型的预测值和真实值的不一致程度)，通过最小化损失来驱动网络的训练。

网络的损失通过前向操作计算；网络参数相对于损失函数的梯度则通过反向操作计算。

分类任务损失：交叉熵损失。

回归任务损失：L1 损失、L2 损失。

浅层神经网络 (Shallow Neural Network) 是神经网络中最基础的形式，通常由一个输入层、一个隐藏层和一个输出层组成。它是一种用于模式识别、分类和回归分析的计算模型，模拟人脑神经元的结构和功能。以下是对浅层神经网络的直观理解：

输入层：这是网络接收输入数据的地方。每个神经元对

应一个输入特征，输入层的任务是将输入数据传递给隐藏层。

隐藏层：这是网络的核心部分，包含多个神经元(节点)。每个神经元接收来自输入层的信号，进行加权求和，然后通过激活函数引入非线性。激活函数(如 Sigmoid、ReLU)使网络能够学习和模拟复杂的非线性关系。

输出层：这是网络产生输出结果的地方。输出层的神经元数量取决于任务的类型，例如分类任务可能有一个或多个神经元对应不同的类别，回归任务可能有一个神经元输出连续值。

权重和偏置：网络中的每个连接都有一个权重值，表示该连接的重要性。偏置项允许模型在拟合数据时有更多的灵活性。在训练过程中，网络通过调整权重和偏置来最小化预测误差。

前向传播：输入数据通过网络逐层传递，每层的输出作为下一层的输入，直到最终输出层产生结果。

反向传播：这是训练网络的过程，通过计算损失函数(预测值与真实值之间的差异)的梯度，调整网络的权重和偏置，使得网络的预测更接近真实值。

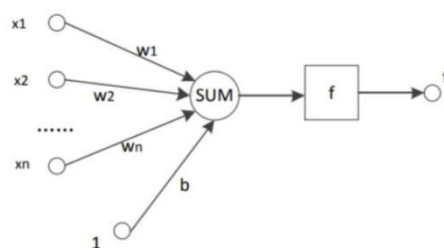
前馈神经网络(FNN，即上层的输出作为下层的输入)是一种单向多层的网络结构，即信息是从输入层开始，逐层向一个方向传递，一直到输出层结束。所谓的“前馈”是指输入信号的传播方向为前向，在此过程中并不调整各层的权

值参数，而反传播时是将误差逐层向后传递，从而实现使用权值参数对特征的记忆，即通过反向传播 (BP) 算法来计算各层网络中神经元之间边的权重。BP 算法具有非线性映射能力，理论上可逼近任意连续函数，从而实现对模型的学习。

感知器：是一种结构最简单的前馈神经网络，也称为感知机，它主要用于求解分类问题。

一个感知器可以接收 n 个输入 $x=(x_1, x_2, \dots, x_n)$ ，对应 n 个权值 $w=(w_1, w_2, \dots, w_n)$ ，此外还有一个偏置项阈值，就是图中的 b 。神经元将所有输入参数与对应权值进行加权求和，得到的结果经过激活函数变换后输出，计算公式如下：

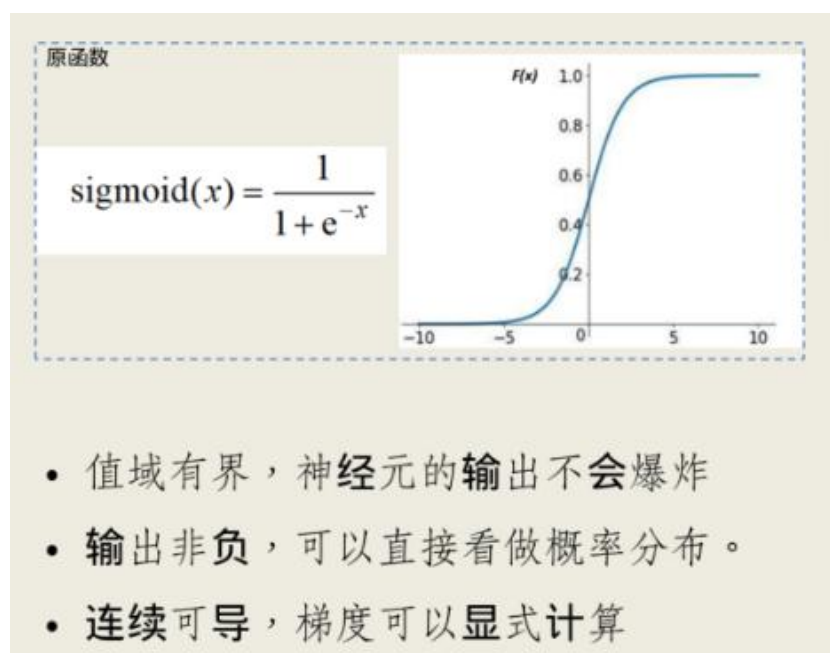
$$y = f(x*w+b)$$



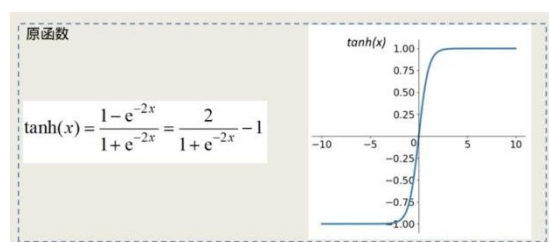
激活函数：如果不用激活函数，每一层输出都是上层输入的线性函数，无论神经网络有多少层，输出都是输入的线性组合。如果使用的話，激活函数给神经元引入了非线性因素，使得神经网络可以任意逼近任何非线性函数，这样神经网络就可以应用到众多的非线性模型中。激活函数的主要作用是提供网络的非线性建模能力。如果没有激活函数，那么该网络仅能够表达线性映射，此时即便有再多的隐藏层，其

整个网络跟单层神经网络也是等价的。因此也可以认为，只有加入了激活函数之后，深度神经网络才具备了分层的非线性映射学习能力。

Sigmoid:

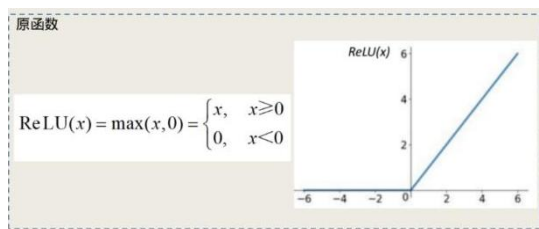


Tanh: 完全可微分的，反对称，对称中心在原点。



继承了 sigmoid 函数一系列优缺点，区别在于 Tanh 函数更适合让神经元产生与输入符号一致的非概率输出。

Relu: 修正线性单元; 保留了 step 函数的生物学启发(只有输入超出阈值时神经元才激活); 函数形式简单，正数时不存在梯度饱和; 一旦输入到了负数，Relu 就会死掉。



分类问题的损失函数：交叉熵

$$J = -\frac{1}{N} \sum_1^N \sum_{i=1}^k y_i \log(p_i)$$

符号解释：

P_i ：某个样本属于第 i 个类别的概率；

y_i ：某个样本的真实类为第 i 类；

示例：有三个类别，I: (1, 0, 0)、II: (0, 1, 0)、III: (0, 0, 1)。

假定样本 x 属于第 2 类，输出的概率值为 0.1, 0.8, 0.1，则其对应的交叉熵为：

$J = -(0 \log(0.1) + 1 \log(0.8) + 0 \log(0.1))$ ， J 越小损失越少。

交叉熵公式的核心思想是：通过比较预测概率分布与真实标签分布的差异，来衡量模型的预测质量。

1. 单样本多分类场景（类别数为 k ）

公式:

$$L = - \sum_{i=1}^k y_i \log(p_i)$$

- y_i : 真实标签的**独热编码**（如第 i 类为真实类别时, $y_i = 1$, 其余为 0）。
- p_i : 模型预测样本属于第 i 类的概率。

2. 批量多分类场景（样本数为 N , 类别数为 k ）

公式:

$$J = - \frac{1}{N} \sum_{j=1}^N \sum_{i=1}^k y_{ji} \log(p_{ji})$$

- N : 批量中的样本数量。
- y_{ji} : 第 j 个样本在第 i 类的真实标签（独热编码）。
- p_{ji} : 第 j 个样本属于第 i 类的预测概率。
- $\frac{1}{N}$: 对批量样本的损失取平均。

核心意义

交叉熵损失通过“对数惩罚”让模型更关注**真实类别对应的预测概率**：若预测概率越接近 1（即模型对真实类别越确信），损失越小；若预测概率接近 0，损失会急剧增大，从而迫使模型优化权重以提升对真实类别的预测置信度。

Python构建基本神经网络

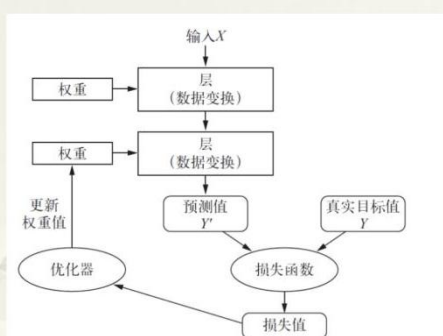


图 神经网络运行原理：层、损失函数和优化器之间的关系

输入数据 (x) 与输出目标 (y)；
层: 多个层组合成网络，**深度学习的基础组件**；
损失函数：均方误差（回归），交叉损失（分类）；
优化器：梯度下降法；

深度学习(神经网络)接收的数据结构——张量(tensor)

一般来说，目前所有的机器学习系统都是用张量作为基本数据结构。

张量：张量就是一个数据容器，其包含的数据通常是数值数据，张量是矩阵向任意维度的推广，张量的维度通常称

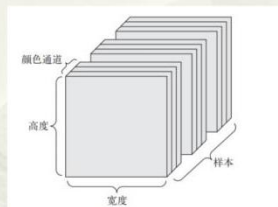
为轴 (axis)。

张量包括：0 阶张量 (标量)；1 阶张量 (向量)；2 阶张量 (矩阵，行观测)；3 阶张量 (图像、时序数据)；...；n 阶张量。

张量在 Python 中通常用 Numpy 数组表示，ndarray。

(3) 3阶张量-图片数据

将多个矩阵打包成一个新的数组称为3阶张量，常用的3阶张量有图像数据。
(width, height, channel), 图像的宽度、高度、深度 (通道数)，彩色图像通道数为3，RGB；灰度图像通道数为1。



(samples, width, height, channel)

图像维度的示意图，如果是多张图像就要算上一个样本维度，此时多幅图像就成为一个4D张量

KerasAPI:

1. Sequential 模型:

- `Sequential` 模型是层的线性堆叠，是用于简单的层堆叠的常见模型。可以通过添加层的方式构建模型。

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(units=64, activation='relu', input_dim=100))
model.add(Dense(units=10, activation='softmax'))
```

2. 函数式 API:

- `tf.keras.Model` 类提供了更灵活的模型构建方式，允许定义具有多输入和多输出的模型，以及共享层的模型。

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
input_layer = Input(shape=(100,))
x = Dense(units=64, activation='relu')(input_layer)
output_layer = Dense(units=10, activation='softmax')(x)
model = Model(inputs=input_layer, outputs=output_layer)
```

3. 层 (Layers):

- Keras 提供了各种层，如全连接层、卷积层、循环层等。这些层可以通过简单的方式添加到模型中。

```
from tensorflow.keras.layers import Dense, Conv2D, LSTM
```

```
4. 损失函数 ( Loss Functions ) :  
- 损失函数用于衡量模型预测与实际值之间的差异。Keras 提供了各种损失函数，  
如均方误差 ( Mean Squared Error )、交叉熵 ( Cross Entropy ) 等。  
from tensorflow.keras.losses import MeanSquaredError,  
SparseCategoricalCrossentropy  
5. 优化器 ( Optimizers ) :  
- 优化器用于更新模型的权重，以最小化损失函数。Keras 提供了 SGD、Adam、  
RMSprop 等优化器。  
from tensorflow.keras.optimizers import SGD, Adam, RMSprop  
6. 指标 ( Metrics ) :  
- 指标用于评估模型性能。Keras 提供了多种指标，如准确率、均方误差等。  
from tensorflow.keras.metrics import Accuracy, MeanSquaredError
```

one-hot 编码：又称为一位有效编码，主要是采用 N 位状态寄存器来对 N 个状态进行编码，每个状态都由独立的寄存器位，并且在任意时候只有一位有效。

图像数据预处理

将数据输入神经网络之前，应该将数据格式化为经过预处理的浮点数张量。数据以 JPEG 文件的形式保存在硬盘中，数据预处理步骤如下：

- 读取图像文件；
- 将 JPEG 文件解码为 RGB 像素网格；
- 将这些像素网格转换为浮点数张量；
- 将像素值（0-255 范围内）缩放到 [0, 1] 区间（神经网络喜欢处理较小的输入值）。

以上功能的实现可借助于 `keras.preprocessing.image`，其包含 `ImageDataGenerator` 类。

image_dataset_from_directory函数介绍

`image_dataset_from_directory()` 是 Keras 提供的一个便捷函数，用于从文件目录中加载图像数据并生成 `tf.data.Dataset` 对象，方便用于模型训练和验证。

函数原型

```
Python 复制

image_dataset_from_directory(
    directory, # 数据文件夹路径
    labels="inferred", # 标签来源
    label_mode="int", # 标签编码方式
    class_names=None, # 类别名称列表
    color_mode="rgb", # 图像颜色模式
    batch_size=32, # 每个批次的图像数量
    image_size=(256, 256), # 图像大小
    shuffle=True, # 是否随机打乱数据
    seed=None, # 随机种子
    validation_split=None, # 验证集划分比例
    subset=None, # 数据子集类型（训练集或验证集）
    interpolation="bilinear", # 图像插值方式
    follow_links=False, # 是否跟随符号链接
    crop_to_aspect_ratio=False # 是否裁剪为宽高比
)
```

数据增强：过拟合的原因是学习样本太少，导致无法训练出能够泛化到新数据的模型。如果拥有无限的数据，那么模型能够观察到数据分布的所有内容，这样就永远不会过拟合。数据增强是从现有的训练样本中生成更多的训练数据，其方法是利用多种能够生成可信图像的随机变换（比如让图像旋转、缩放等）来增加样本。其目标是，模型在训练时不会两次查看完全相同的图像。这让模型能够观察到数据的更多内容，从而具有更好的泛化能力。在 Keras 中，这可以通过对 `ImageDataGenerator` 实例读取的图像执行多次随机变换来实现。

回调函数 (callbacks): Keras 中回调函数的作用就是动态检测模型的运行过程，主要包括：

(1) 模型检查点 (model checkpointing): 在训练过程中的不同时间点保存模型的当前状态；

(2) 提前终止 (early stopping): 如果验证损失不再改善，则中断训练，同时保存在训练过程中的最佳模型；

(3) 在训练过程中动态调节某些参数值：比如调节优化器的学习率；

(4) 在训练过程中记录训练指标和验证指标，或者将模型学到的表示可视化（这些表示在不断更新）：`fit()` 进度条实际上就是一个回调函数。

使用预训练的卷积神经网络：想要将深度学习应用于小型图像数据集，一种常用且非常高效的方法是使用预训练网络。预训练网络（`pretrained network`）是一个保存好的网络，之前已在大型数据集（通常是大规模图像分类任务）上训练好。如果这个原始数据集足够大且足够通用，那么预训练网络学到的特征的空间层次结构可以有效地作为视觉世界的通用模型，因此这些特征可用于各种不同的计算机视觉问题，即使这些新问题涉及的类别和原始任务完全不同。举个例子，你在 ImageNet 上训练了一个网络（其类别主要是动物和日常用品），然后将这个训练好的网络应用于某个不相干的任务，比如在图像中识别家具。这种学到的特征在不同问题之间的可移植性，是深度学习与许多早期浅层学习方法相比的重要优势，它使得深度学习对小数据问题非常有效。

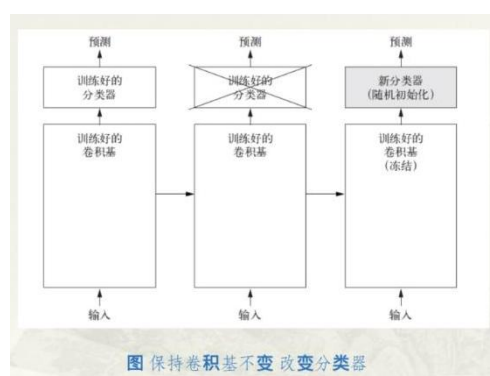
（迁移学习）

预训练的神经网络包括：特征提取和模型微调。

1. 特征提取：特征提取是使用之前网络学到的表示（权

重) 来从新样本中提取出有趣的特征。然后将这些特征输入一个新的分类器, 从头开始训练。如前所述, 用于图像分类的卷积神经网络包含两部分: 首先是一系列池化层和卷积层, 最后是一个密集连接分类器。第一部分叫作模型的卷积基(convolutional base)。对于卷积神经网络而言, 特征提取就是取出之前训练好的网络的卷积基, 在上面运行新数据, 然后在输出上面训练一个新的分类器。

注意: 某个卷积层提取的表示的通用性(以及可复用性)取决于该层在模型中的深度, 模型中更靠近底部的层提取的是局部的、高度通用的特征图(比如视觉边缘、颜色和纹理), 而更靠近顶部的层提取的是更加抽象的概念(比如“猫耳朵”或“狗眼睛”)。因此, 如果你的新数据集与原始模型训练的数据集有很大差异, 那么最好只使用模型的前几层来做特征提取, 而不是使用整个卷积基。



为什么仅重复使用卷积基? 我们能否也重复使用密集连接分类器? 一般来说, 应该避免这么做。原因在于卷积基学到的表示可能更加通用, 因此更适合重复使用。卷积神经网络的特征图表示通用概念在图像中是否存在, 无论面对什

么样的计算机视觉问题，这种特征图都可能很有用。但是，分类器学到的表示必然是针对于模型训练类别，其中仅包含某个类别出现在整张图像中的概率信息。

在编译和训练模型之前，一定要“冻结”卷积基。冻结（freeze）一个或多个层是指在训练过程中保持其权重不变。如果不这么做，那么卷积基之前学到的表示将会在训练过程中被修改。因为其上添加的 Dense 层是随机初始化的，所以非常大的权重更新将会在网络中传播，对之前学到的表示造成很大破坏。

2. 模型微调：另一种广泛使用的模型复用方法是模型微调（fine-tuning），与特征提取互为补充；微调是指将卷积基顶部的几层“解冻”，并将解冻的几层和新增的分类器部分联合训练。之所以称为“微调”，是因为它仅略微调整了所复用模型中更加抽象的表示（顶层提取的特征更抽象，如猫的眼睛、狗的耳朵等），从而让这些表示与当前任务更相关。

序列数据及相关任务

1. 文本序列

定义：语言和文字，说的话、写的文章、诗歌等，属于自然语言处理（NLP）；

特点：长度不等、前后相依；

任务：

文本分类：该文本的主题是什么；

内容过滤：文本是否含有脏话、是否为欺诈信息；

情感分析：电影评论；

语言模型：人机交互、文字接龙；

机器翻译：英译汉等。

2. 时间序列数据

定义：定期测量获得的任意数据，例如每日股价、城市每小时耗电量、商店每周销售额；自然现象（地震活动、鱼类种群演变、某地天气）、人类活动模式（网站访问者、国家 GDP、信用卡交易）等场景均存在时间序列；

处理特点：需了解系统的动力学（dynamics），包括系统的周期性循环、随时间变化规律、周期规律与突然激增等；

任务：

回归与分类预测：股票收益预测、天气状况预测（气温、是否下雨）；

事件检测：识别连续数据流中特定预期事件的发生；

异常检测：检测连续数据流中出现的异常情况。

RNN基本原理

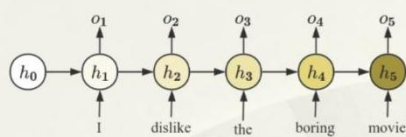


图 展开的RNN模型

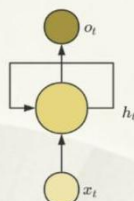


图 折叠的RNN模型

在每个时间 t ，网络接受当前时间输入 x_t 和上一时刻状态向量 h_{t-1} ，经过

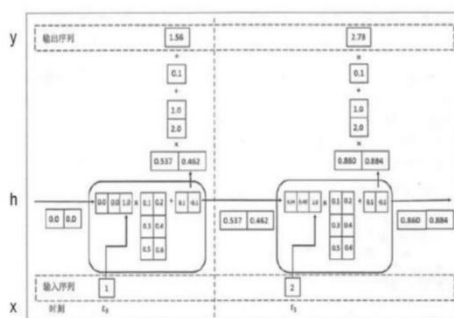
$$h_t = f_W(h_{t-1}, x_t)$$

变换后得到当前时刻新的状态向量 h_t ，并写入内存， W 为网络的参数集，在每个时间 t 上网络层均有输出产生 O_t ， $O_t = g(h_t)$ 即网络状态向量 h_t 变换后的输出。

RNN定义：如上图所示，网络循环接受序列的每个特征向量 x_t ，并刷新内部状态向量 h_t ，同时形成输出 O_t ，对于这种网络结构将其称之为“循环神经网络”（Recurrent Neural Network）。

RNN基本原理

RNN示例



$$h_t = \tanh(h_{t-1}w_{hh} + x_t w_{xh} + b_h)$$

$$= \tanh([h_{t-1}, x_t]W + b_h)$$

$$y_t = \text{softmax}(w_{hy} h_t + b_y)$$

上一时刻的状态（记忆）与当前时刻的输入拼接成一维向量作为循环体的全连接层神经网络的输入。

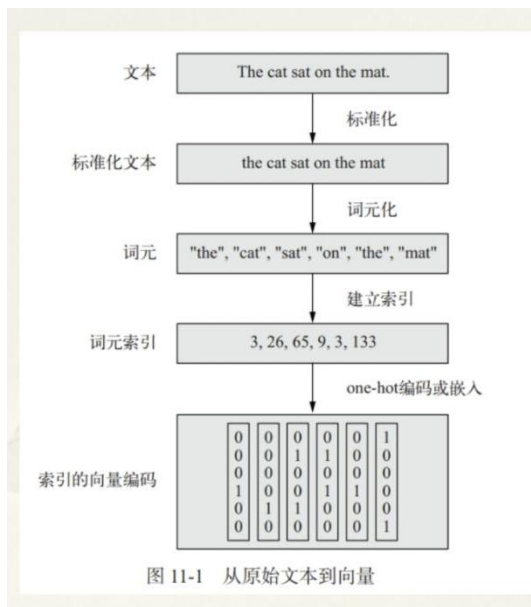
关于 RNN 网络的一些说明：

和卷积神经网络一样，循环神经网络虽然在时间轴上展开了多次，但只能算一个网络。通过在深度方向堆叠多个 SimpleRNN 类实现与深度卷积神经网络同样的效果，提升网络的表达能力。与 CNN 相比动辄几十层、上百层，循环神经网络很容易出现“梯度发散”与“梯度爆炸”，深层的循环神经网络训练起来非常困难，目前 RNN 的层数一般控制在 10 层以内。

文本数据的预处理

处理流程与实现方式

流程：文本 → 数值张量 → 文本向量化（vectorize）
（文本向量化环节会反馈作用于数值张量）。



Word2vec 算法介绍

通过利用语料库中的上下文信息完成训练，将 one-hot 编码的向量嵌入到维度较低的空间中实现单词向量化，后续可利用训练得到的词向量完成文本处理相关任务。解决了独热（one-hot）编码无法发现语义信息、向量维度过大过稀疏的缺点。

模型结构：

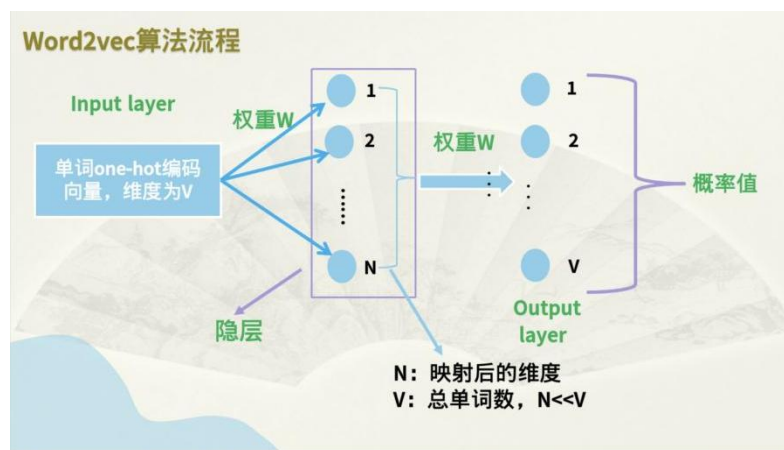
是一个三层神经网络模型，包含输入层、隐藏层、输出

层。

若训练数据词汇表大小为 V ，期望词嵌入向量维度为 N ，则输入层和输出层各有 V 个神经元，隐藏层有 N 个神经元。

输入层输入的是 one-hot 编码向量（维度为 V ），仅一个维度值为 1）；隐藏层无激活函数，为线性单元；输出层输出经 SoftMax 处理的结果，网络为全连接结构。

训练后得到词嵌入矩阵，隐藏层的输出即为通过 Word2Vec 模型计算得到的词向量。



长短记忆神经网络(LSTM)传统 RNN 的核心问题传统 RNN 在时刻 t 理论上应记住多个时间步前的信息，但实际无法学习长期依赖关系，根源是“梯度消失”：

$$Dw = h \frac{\partial E}{\partial W}$$

无变化导致网络难以训练，即权重 $w(t)$ 到 $w(t+1)$ 的变化极小。

LSTM 的改进机制：

LSTM 引入了“跨时间步携带信息”的方法，可类比传

送带：序列中的信息可在任意位置“跳上传送带”，并在更晚的时间步“跳回”，实现信息的原封不动复用。

核心作用：保存早期信息供后续使用，防止早期信号在处理中逐渐消失。

RNN 短时记忆

RNN 的短时记忆问题：

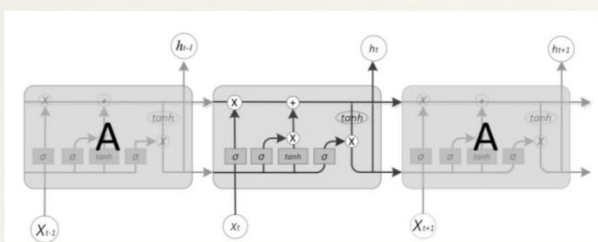
RNN 除训练困难外，还存在短时记忆的严重问题。以长句为例：“今天天气太美好，尽管路上发生了一些不愉快……我马上调整好状态，开开心心地准备迎接美好的一天。”人类能理解“开开心心”与句首“今天天气美好”的关联，但 RNN 处理长句时，往往只能理解有限长度内的信息，对较长范围内的有用信息无法很好利用，这种现象称为短时记忆。

LSTM 的提出与优势

为延长短时记忆、让 RNN 有效利用大范围训练数据以提升性能，提出长短时记忆网络（LSTM）。

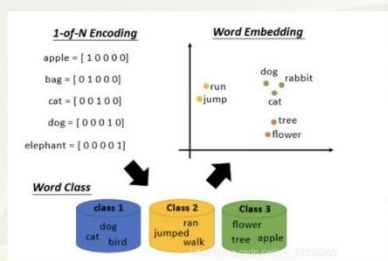
相较于传统 RNN，LSTM 记忆能力更强，更擅长处理长序列信号，被广泛应用于序列预测、自然语言处理领域，几乎取代了传统 RNN。

LSTM原理



LSTM新增了一个状态向量 C_t ,并引入了门控机制,通过门控单元来控制信息的遗忘和刷新。在LSTM中有两个状态向量 c 和 h , c 作为LSTM内部状态向量即LSTM内存状态向量Memory,而 h 表示LSTM输出向量。相对与传统RNN,LSTM把内容Memory和输出分为两个变量,利用3个门控:输入门、遗忘门(forget gate)和输出门来控制内部信息的流动。

单词嵌入 (word embedding)



(1) 1-of-N encoding (one-hot encoding): 每一个词汇都当做一个符号,都用向量来描述,这个方法是有不足的,这样词汇和词汇之间的相关性反映不出来,而且过于稀疏。

(2) Word Class: 建立word class,把有相同性质的word放在同一个class内,将词汇进行分类,这个方法也比较粗糙,比如动物也分了很多种,不能完全概况。

(3) Word Embedding: 每一个的词汇也用向量来描述,但是每一个维度是一个属性。word embedding,就是找到一个映射或者函数,生成在一个新的空间上的表达。通俗的翻译可以认为是单词嵌入,就是把X所属空间的单词映射为到Y空间的多维向量,那么该多维向量相当于嵌入到Y所属空间中。与one-hot编码和word class相比,词嵌入可以将更多的信息塞入更低的维度中。

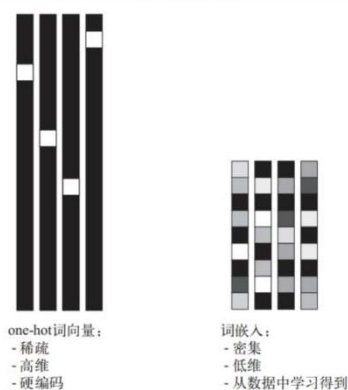


图 11-2 one-hot 编码或 one-hot 哈希得到的词表示是稀疏、高维、硬编码的,而词嵌入是密集、相对低维的,而且是从数据中学习得到的

keras 的 Embedding 层介绍

代码示例:

```
embedding_layer=layers.Embedding(input_dim=max_tokens,output_dim=256)
```

核心参数与功能:

必选参数：词元个数（`input_dim`）和嵌入维度（`output_dim`）。

功能类比：可理解为“字典”，将整数索引（表示单词）映射为密集向量；输入整数，内部查询后返回对应向量，本质是“字典查询”，流程如图 11-4 所示。

张量形状说明：

输入：形状为（`batch-size`, `sequence-length`）的 2 阶整数张量，元素为整数序列。

输出：形状为（`batch-size`, `sequence-length`, `embedding-dimensionality`）的 3 阶浮点数张量。

建立多任务深度学习模型：

定义：多任务学习（MTL）是深度学习领域中的一种重要方法，其核心思想是同时学习多个相关任务，并通过共享表示来提高各个任务的学习效率和性能。

该方法与传统的单任务学习（STL）形成对比，后者通常针对每个任务独立地训练一个模型。

优势

1. 参数共享：通过在多个任务之间共享网络层，可减少模型的参数数量，从而降低过拟合风险，提升模型泛化能力。

2. 信息互补：不同任务之间可能存在共性，共享表示能帮助模型将从一个任务中学习到的知识迁移到其他任务，提

高学习效率。

3. 数据效率：在数据稀缺的场景下，多任务学习可通过跨任务的知识转移来提高模型性能。

4. 计算资源优化：同时训练多个任务能更高效地利用计算资源，尤其在分布式计算环境中表现突出。

深度学习要点总结

1. Keras 图像输入形状：

利用 Python-Keras 编程时，若一个尺寸为 $a*b$ 的图像输入到全连接层，需将其拉直为 $\text{shape}=(a*b)$ ；若输入到卷积层，无需拉直，保持 $\text{shape}=(a, b)$ 即可。

2. 卷积神经网络（CNN）的卷积核：

卷积核中的元素是卷积层的权重，在 CNN 中通过学习过程优化得到，用于对输入数据执行卷积操作。

卷积核的大小和形状在网络设计阶段确定，决定了卷积操作的特征提取方式。

卷积操作逻辑：在输入数据的不同位置应用卷积核，将权重与对应位置的输入值相乘，再将所有乘积相加得到输出值。卷积核的每个元素都是一个权重，用于对输入数据的特定位置加权，这些权重由模型通过训练学习得到，从而提取有用特征。

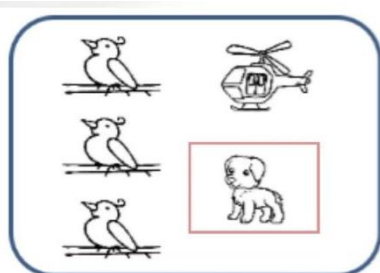
3. Keras 图像与标签处理：

利用 Keras 将图像转换为像素值时，拟合模型务必对类

别标签进行 one-hot 编码。

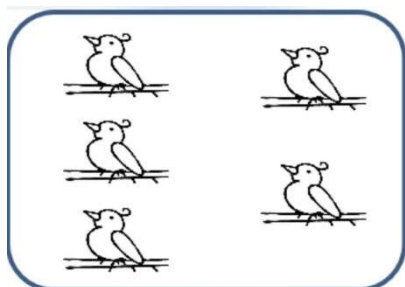
信息熵公式：

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$



$$p1 = \frac{3}{5}, p2 = \frac{1}{5}, p3 = \frac{1}{5}$$

$$H = -\frac{3}{5} \log(\frac{3}{5}) - \frac{1}{5} \log(\frac{1}{5}) - \frac{1}{5} \log(\frac{1}{5}) = 0.9502$$



$$p1 = \frac{5}{5}$$

$$H = -\frac{5}{5} \log(\frac{5}{5}) = 0.0$$

总结：数据越纯净，信息熵的值越小。

准确率、精准率和召回率：

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

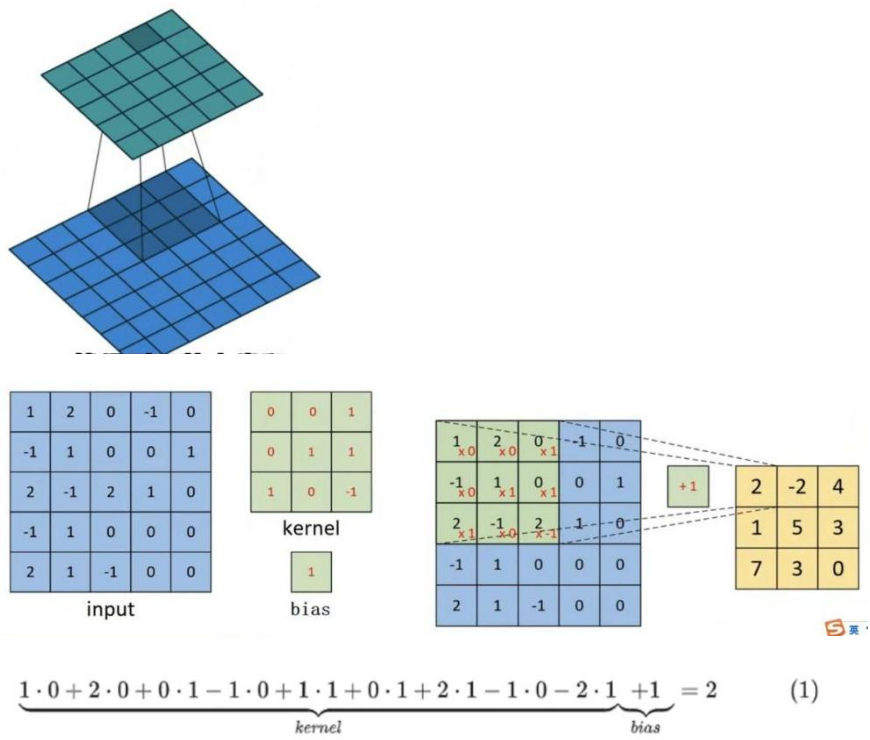
$$Precision = \frac{TP}{TP + FP}$$

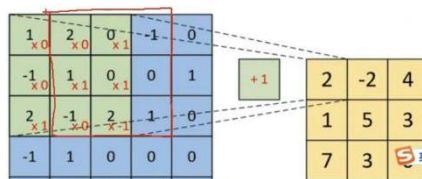
$$Recall = \frac{TP}{TP + FN}$$

- True Positive (TP) : 表示将正样本正确的预测为了正样本;
- False Positive (FP) : 表示将负样本错误的预测为了正样本;
- False Negative (FN) : 表示将正样本错误的预测为了负样本;
- True Negative (TN) : 表示将负样本正确的预测为负样本;

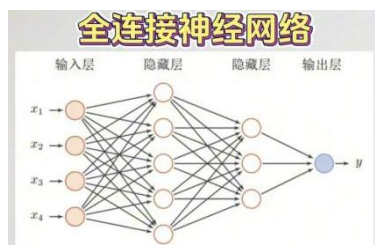
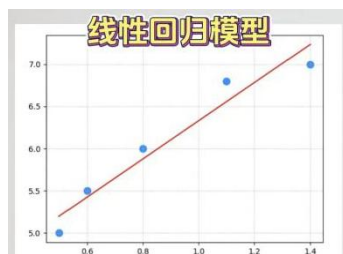
卷积：每次取一个特定大小的矩阵 F，然后将其对输入矩阵 X 依次扫描，进行内积运算的过程。每移动一个位置，就会计算得到一个卷积值，当 F 扫描完，就得到了整个卷积

后的结果 Y。

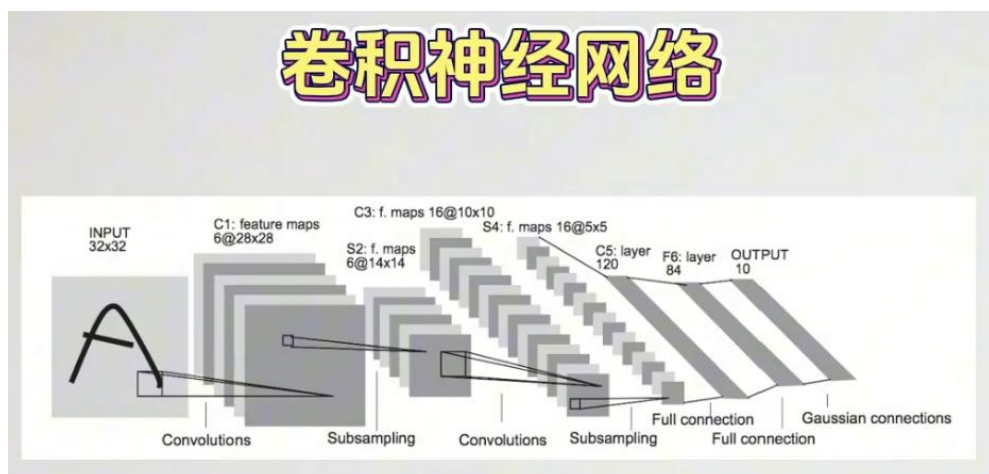




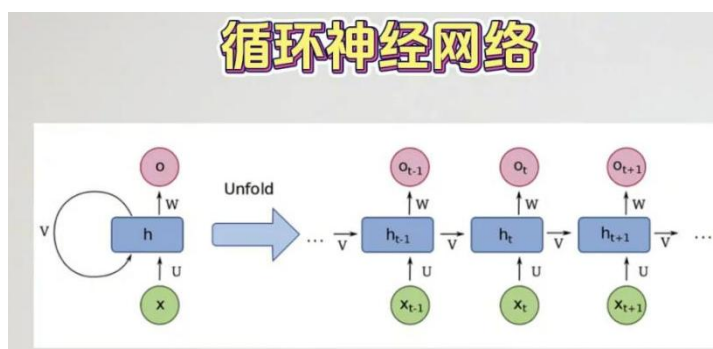
五大核心:

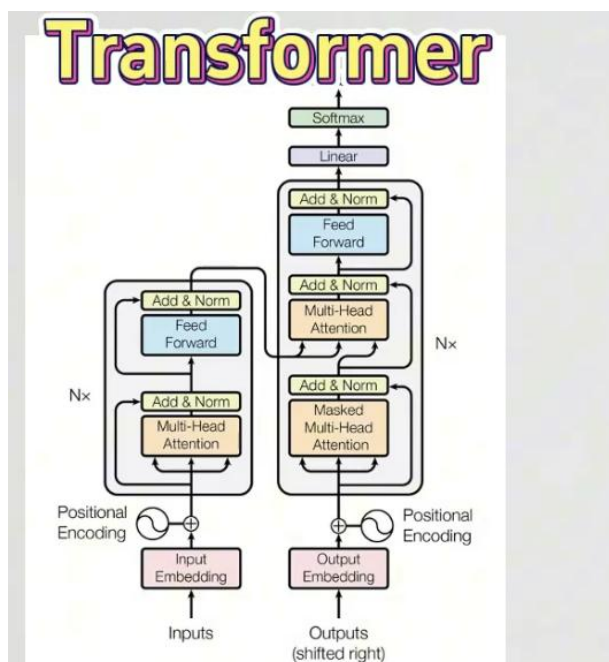


卷积神经网络



循环神经网络





Transformer 中的注意力机制：

当你看到“苹果”这个词的时候，会想到下面的哪种？



当“苹果”这个词出现在“买一个苹果手机”这句话中时，这句话里面的其他的词哪个对苹果的理解影响比较大，没错，是手机。因为手机直接决定了这里指的是品牌。

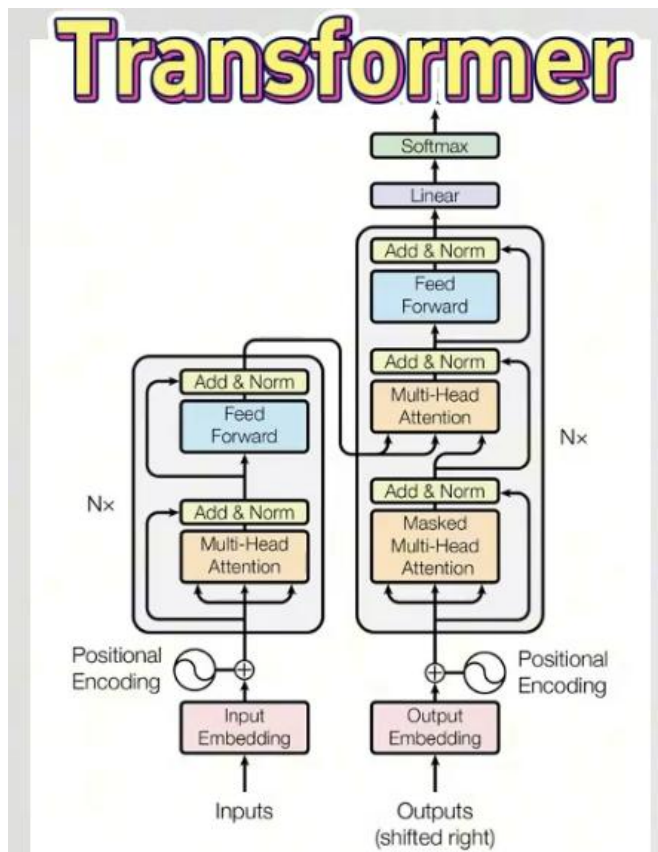
Transformer 中的注意力机制也是类似的。

为了理解苹果在这句话中的意思，Transformer 会对每个词分配不同的注意力分数，其实就是权重，比如手机的权重较高，因为它对苹果的影响最大。

买	0.2
一个	0.1
苹果	0.3
手机	0.4

而这个词都会根据它的权重重新被理解，然后得到一个当前的结果，把所有词的理解结合到一起，再结合你对原本“苹果”这个词的认知，就能得到苹果在这句话当中最终的意思就是苹果手机。

如果觉得理解的还不够，就继续叠加多层，再次的对这句话当中的词进行一个理解，这就是 Transformer 当中的 N。也就是 Transformer 当中它叠加了多层注意力，层数越多，模型对你输入的句子理解也就越准确。



Transformer 中的四种注意力机制:



计算卷积后输出特征图的大小，可使用以下公式：

$$\frac{\text{输入大小} - \text{卷积核大小} + 2 \times \text{填充}}{\text{步长}} + 1$$

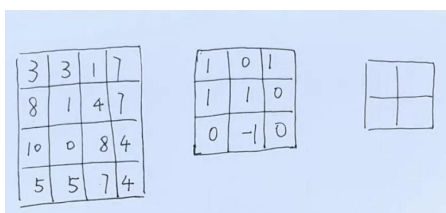
输入大小：输入特征图的边长（示例中输入是 4×4 ）。

卷积核大小：卷积核的边长（示例中是 3×3 ）。

填充（padding）：在输入特征图周围补充的层数（如填充 0 表示不补充，填充 1 表示周围各补 1 层）。

步长（stride）：卷积核每次滑动的步幅。

步长为 1 的情况下，输出特征图为 2×2 ，与图中右侧的空矩阵尺寸对应。



卷积是为了提取图像中的细节特征；池化是为了方便简化这些特征。

常见方式：最大池化和平均池化。

池化的步长和窗口的大小是一致的。

池化

1	2	3	4	5	6
-1	0	1	1	0	2
0	1	0	-1	0	0
-1	0	1	0	1	2
0	2	1	0	4	1
2	1	-2	2	-1	-2

池化

最大池化:

2	4	6
1	1	2
2	2	4

平均池化:

0.5	2.25	3.25
0	0	0.75
1.25	0.25	0.5

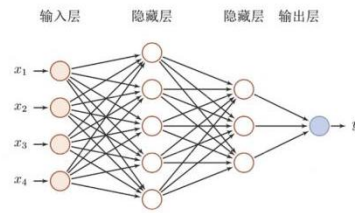
池化的窗口是没有参数的，所以在训练的时候不需要更新这里边的参数数据。

超参数：由开发者在模型训练之前设计好的。

模型参数：模型在训练过程中通过数据学习得到的。

神经网络

$$y = (f^3(f^2(f^1(x))))$$

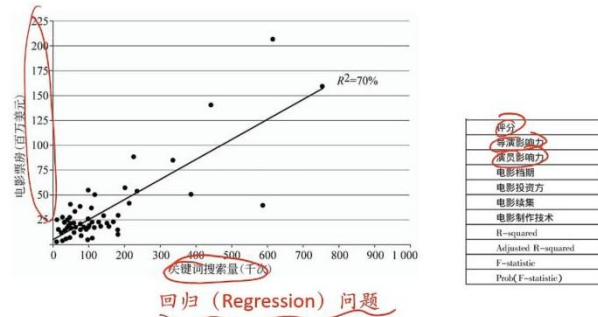


机器学习 ≈ 自动构建一个映射函数

语音识别	$f(\text{语音信号}) = \text{"你好"}$
图像识别	$f(\text{图像}) = \text{"猫"}$
围棋	$f(\text{棋局}) = \text{"5-5" (落子位置)}$
对话系统	$f(\text{用户输入}) = \text{机器输出}$

回归问题：输出是一个连续值。

示例：电影票房预测



常见的机器学习类型

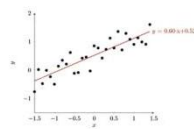
	监督学习	无监督学习	强化学习
训练样本	训练集 $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$	训练集 $\{\mathbf{x}^{(n)}\}_{n=1}^N$	智能体和环境交互的 轨迹 τ 和累积奖励 G_τ
优化目标	$y = f(\mathbf{x})$ 或 $p(y \mathbf{x})$	$p(\mathbf{x})$ 或带隐变量 $\mathbf{z}$ 的 $p(\mathbf{x} \mathbf{z})$	期望总回报 $\mathbb{E}_\tau[G_\tau]$
学习准则	期望风险最小化 最大似然估计	最大似然估计 最小重构错误	策略评估 策略改进

示例: AlphaGo

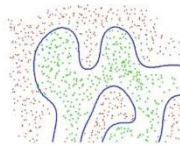


强化学习-通过与环境进行交互来学习

典型的监督学习问题

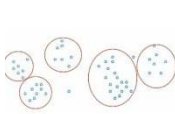


回归

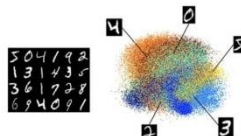


分类

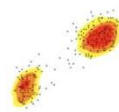
典型的无监督学习问题



聚类



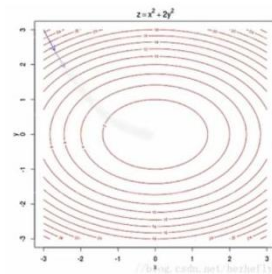
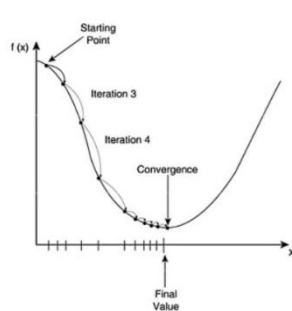
降维



密度估计

损失函数: 是一个非负实数函数, 用来量化模型预测和真实标签之间的差异。

梯度下降法 (Gradient Descent)



$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha \frac{\partial \mathcal{R}_D(\theta)}{\partial \theta} \\ &= \theta_t - \alpha \frac{1}{N} \sum_{n=1}^N \frac{\partial \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \theta))}{\partial \theta}\end{aligned}$$

搜索步长 α 中也叫作**学习率** (Learning Rate)

随机梯度下降法 (Stochastic Gradient Descent, SGD)

▶ 随机梯度下降法：在每次迭代时只采集一个样本

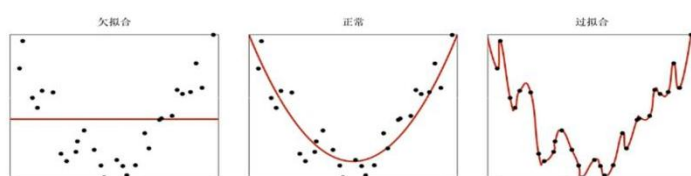
$$\theta_{t+1} = \theta_t - \alpha \frac{\partial \mathcal{L}(y^{(n)}, f(x^{(n)}; \theta))}{\partial \theta} \quad n = 1 \dots N$$

▶ 当经过足够次数的迭代时，随机梯度下降也可以收敛到局部最优解

优点是每次计算开销小，支持在线学习。

缺点是无法充分利用计算机的并行计算能力。

小批量随机梯度下降法：随机选取一小部分训练样本来计算梯度并更新参数，既可以兼顾随机梯度下降法的优点，也可以提高训练效率。



过拟合：经验风险最小化原则很容易导致模型在训练集上错误率很低，但是在未知数据上错误率很高。

过拟合问题往往是由于训练数据少和噪声等原因造成的。

经验风险（训练表现）和期望风险（真实泛化表现）通常不相等。因为训练集只是真实数据的“抽样”，存在抽样偏差——训练集的分布无法完全代表真实世界的全局分布。

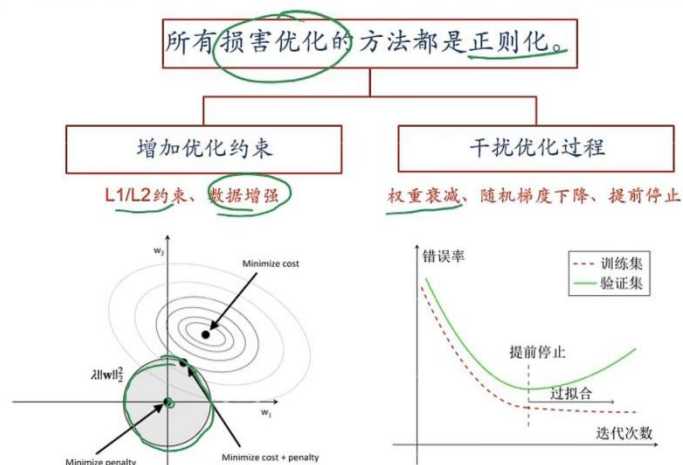
机器学习的核心目标之一，就是让经验风险尽可能接近期望风险，从而缩小“训练表现”与“真实泛化表现”的差距（即降低泛化误差）。这也是正则化、验证集、数据增强等技术的设计初衷。

“为什么模型在训练集上表现好，却可能在测试集上表现差”——因为训练集的经验风险和真实世界的期望风险存在差异，而缩小这种差异就是泛化能力的体现。

如何减少泛化误差：

- ① 优化：经验风险最小。
- ② 正则化：降低模型复杂度。

正则化 (Regularization)



提前停止：使用一个验证集来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。

模型选择时如果遇到数据稀疏问题，可以使用交叉验证的方法：

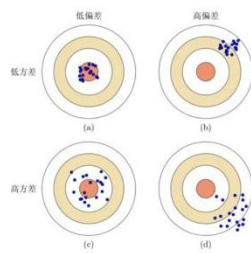
将训练集分成 S 组；

每次用 $S-1$ 组训练模型，剩下 1 组作为验证集评估性能；

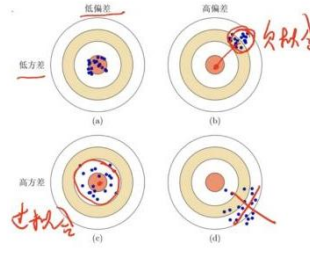
重复 S 次后，取验证集平均性能最好的模型作为最终选择。

作用：避免因“单一验证集划分”导致的评估偏差，尤其在数据稀疏或样本量小时，能更可靠地反映模型在未知数据上的真实表现。

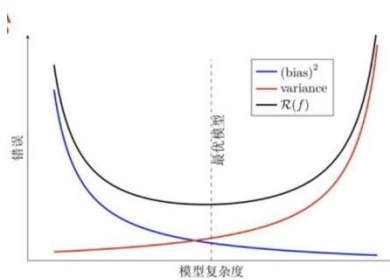
模型选择：偏差与方差



模型选择：偏差与方差



最小化期望风险等价于最小化偏差和方差之和



熵用来衡量一个随机事件的不确定性：熵越高，随机变量的信息越多；熵越低，随机变量的信息越少。

自信息：一个随机事件所包含的信息量。

对于一个随机变量 X ，当 $X=x$ 时的自信息 $I(x)$ 定义为：

$$I(x) = -\log p(x)$$

熵为 0 就代表着其中没有不确定性。

熵编码：在对分布 $p(y)$ 的符号进行编码时，熵 $H(p)$ 也是理论上最优的平均编码长度。概率高就赋予更小的编码长度，概率低赋予更长的编码长度，这样会实现一个最优编码。

交叉熵 (Cross Entropy)

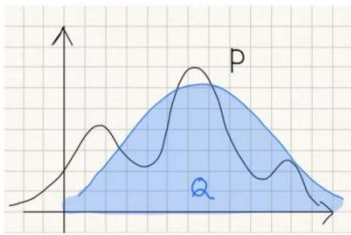
- 交叉熵是按照概率分布 q 的最优编码对真实分布为 p 的信息进行编码的长度。

$$\begin{aligned} H(p, q) &= \mathbb{E}_p[-\log q(x)] \\ &= -\sum_x p(x) \log q(x) \end{aligned}$$

- 在给定 q 的情况下, 如果 p 和 q 越接近, 交叉熵越小;
- 如果 p 和 q 越远, 交叉熵就越大。

KL散度 (Kullback-Leibler Divergence)

- KL散度是用概率分布 q 来近似 p 时所造成的信息损失量。
- KL散度是按照概率分布 q 的最优编码对真实分布为 p 的信息进行编码, 其平均编码长度 (即交叉熵) $H(p, q)$ 和 p 的最优平均编码长度 (即熵) $H(p)$ 之间的差异。

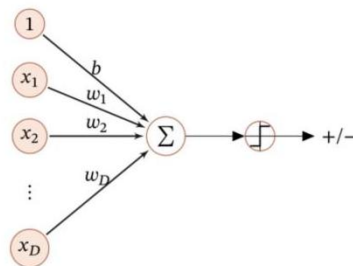


$$\begin{aligned} \text{KL}(p, q) &= H(p, q) - H(p) \\ &= \sum_x p(x) \log \frac{p(x)}{q(x)} \end{aligned}$$

Logistic回归 (Logistic Regression)

- 也称为对数几率回归, 简称对率回归
- 模型

$$g(f(\mathbf{x}; \mathbf{w})) = \begin{cases} 1 & \text{if } f(\mathbf{x}; \mathbf{w}) > 0 \\ 0 & \text{if } f(\mathbf{x}; \mathbf{w}) < 0 \end{cases}$$



Softmax回归

▶ 多分类问题 (Multi-class Classification)

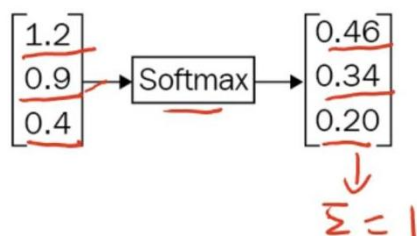
▶ 对于多类问题，类别标签 $y \in \{1, 2, \dots, C\}$ 可以有 C 个取值。

▶ 模型

$$y = \arg \max_{c=1}^C f_c(\mathbf{x}; \mathbf{w}_c)$$

$$f_c(\mathbf{x}; \mathbf{w}_c) = \mathbf{w}_c^T \mathbf{x} + b_c, \quad c \in \{1, \dots, C\}$$

先建立 C 个判别函数，然后对于一个输入的 $\mathbf{x}$ ，把它带到 C 个判别函数中，每一个得到一个打分，然后哪个打分最高，就决策它属于哪一个类。



交叉熵损失是一种最大似然估计。

最大似然估计的思想：在已知观测数据的情况下，寻找一组模型参数，使得这组观测数据出现的“可能性”最大。

交叉熵损失函数

▶ 向量形式

$$-\mathbf{y}^T \log \hat{\mathbf{y}} \quad \mathbf{y} = [I(1=c), I(2=c), \dots, I(C=c)]^T$$

Handwritten notes: $P_\theta(\mathbf{y}|\mathbf{x})$ and one-hot are present next to the equation.

▶ 对于一个三类分类问题，类别为 $[0, 0, 1]$ ，预测类别概率为 $[0.3, 0.3, 0.4]$ ，则交叉熵损失函数为

$$\begin{aligned} \mathcal{L}(\theta) &= -(0 \times \log(0.3) + 0 \times \log(0.3) + 1 \times \log(0.4)) \\ &= -\log(0.4). \end{aligned}$$

在线学习是机器学习的一种方法或框架。在在线学习中，模型的学习过程是连续、顺序和增量的。

其核心特点是：数据是“流式”到达的：模型不是一次性拿到所有数据（批量学习），而是按顺序、一个一个（或一小批一小批）地接收数据样本。

即时学习与预测：模型每收到一个新的数据样本，都会立即进行两个动作：1. 做出预测。2. 根据预测结果收到的反馈（如真实标签），立即更新自身的参数，以改进对下一个样本的预测。

一个生动的比喻：

批量学习（您熟悉的方式）：像学生在考试前，先拿到一本完整的教科书，花几个月时间闭关学习所有章节，然后参加期末考试。考完后知识就固定了。

在线学习：像一位棋手在不断对弈中成长。他每下一盘棋（收到一个数据样本），就会根据这盘棋的胜负结果（反馈），立即反思并调整自己的策略（更新模型），然后以更好的状态迎接下一盘棋。棋局（数据分布）可能千变万化，他必须随时适应。

“错误驱动”更准确的意思是：模型参数的更新幅度和方向，完全由“犯错”的程度（即损失值）和“犯错”的方式（即损失函数的梯度）来决定。几乎没有不犯错就更新的情况。

一个生动的比喻：蒙眼下山

想象你在一座形状复杂的山上蒙眼下山（山的高度代表

损失，你的目标是找到最低点）。

您的理解：“只有当我往上走（犯错）时，我才调整方向。”

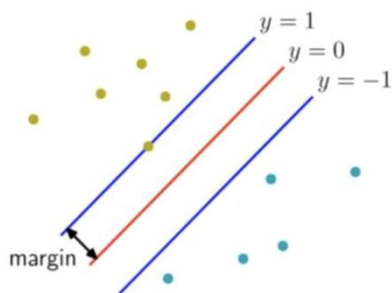
实际的梯度下降：你每秒钟都在用脚感受地面最陡的下坡方向（计算梯度），然后向那个方向迈出一小步（更新参数）。

即使你已经在下降过程中，你仍然在持续感受坡度并调整，以确保自己沿着最速下降路径前进。

这个“感受坡度”的过程，就是计算损失函数的梯度。只要有坡度（梯度不为零），你就会移动（更新）。只有当到达谷底（梯度为零，损失最小）时，你才会停止。

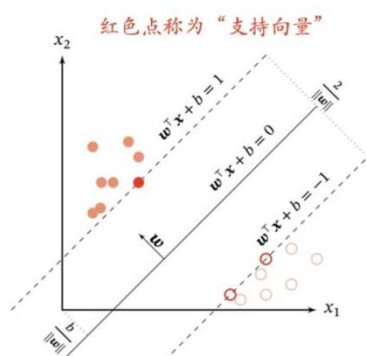
间隔：决策边界到分类样本的最短距离。

支持向量机：选择间隔最大的决策边界。



好的分类器的目标是使得这个间隔最大化。相当于所有的样本离这个分界面的距离都要尽可能大，特别是最短距离也要大。

支持向量机 (Support Vector Machine, SVM)



支持向量机的目标是寻找一个超平面 $(\mathbf{w}^*, b^*)$ 使得 γ 最大, 即

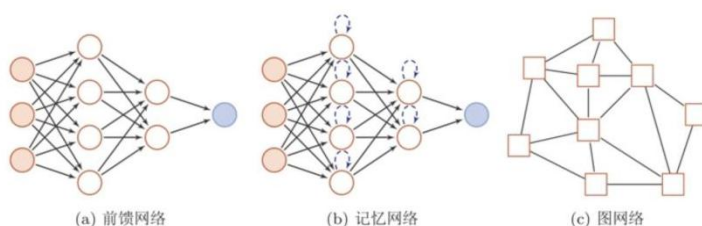
$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \gamma \\ \text{s.t.} \quad & \frac{y^{(n)}(\mathbf{w}^T \mathbf{x}^{(n)} + b)}{\|\mathbf{w}\|} \geq \gamma, \forall n \in \{1, \dots, N\} \end{aligned}$$

数据集 $\mathcal{D}$ 中每个样本 $\mathbf{x}^{(n)}$ 到分割超平面的距离为:

$$\gamma^{(n)} = \frac{|\mathbf{w}^T \mathbf{x}^{(n)} + b|}{\|\mathbf{w}\|} = \frac{y^{(n)}(\mathbf{w}^T \mathbf{x}^{(n)} + b)}{\|\mathbf{w}\|}$$

软间隔: 在支持向量机的优化问题中, 如果训练集中的样本不是线性可分的, 就无法找到最优解。为了能够容忍部分不满足约束的样本, 可以引入松弛变量。

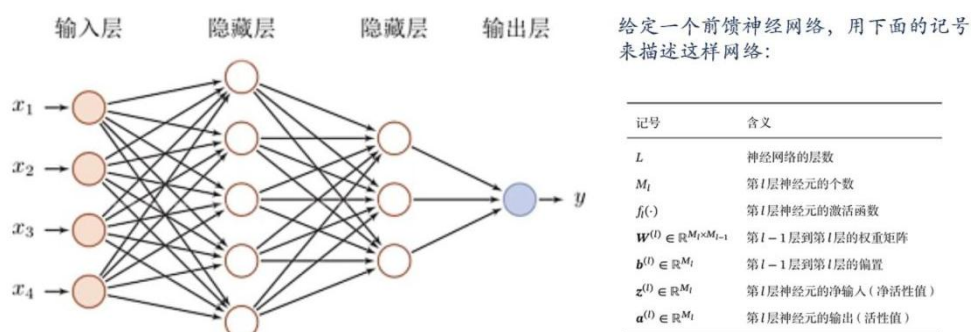
线性模型	激活函数	损失函数	优化方法
线性回归	-	$(y - \mathbf{w}^T \mathbf{x})^2$	最小二乘、梯度下降
Logistic 回归	$\sigma(\mathbf{w}^T \mathbf{x})$	$y \log \sigma(\mathbf{w}^T \mathbf{x})$	梯度下降
Softmax 回归	$\text{softmax}(\mathbf{W}^T \mathbf{x})$	$y \log \text{softmax}(\mathbf{W}^T \mathbf{x})$	梯度下降
感知器	$\text{sgn}(\mathbf{w}^T \mathbf{x})$	$\max(0, -y \mathbf{w}^T \mathbf{x})$	随机梯度下降
支持向量机	$\text{sgn}(\mathbf{w}^T \mathbf{x})$	$\max(0, 1 - y \mathbf{w}^T \mathbf{x})$	二次规划、SMO 等



圆形节点表示一个神经元, 方形节点表示一组神经元。

前馈神经网络 (全连接神经网络、多层感知器): 各神经元分别属于不同的层, 层内无连接。相邻两层之间的神经元全部两两连接、整个网络中无反馈, 信号从输入层向输出层单向传播, 可用一个有向无环图表示。

定义一个网络的层数时，通常来讲不算输入层。因为输入层就是一个接受外界信息的。所以第一个隐藏层称为第一层。下图就是一个三层的网络。有时候把输入层称为零层。



前面三个记号都是提前定义好的，称为超参数。

通用近似定理

定理 4.1 – 通用近似定理 (Universal Approximation Theorem) [Cybenko, 1989; Hornik et al., 1989]: 令 $\phi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathcal{J}_D$ 是一个 D 维的单位超立方体 $[0, 1]^D$, $C(\mathcal{J}_D)$ 是定义在 $\mathcal{J}_D$ 上的连续函数集合. 对于任意给定的一个函数 $f \in C(\mathcal{J}_D)$, 存在一个整数 M , 和一组实数 $v_m, b_m \in \mathbb{R}$ 以及实数向量 $w_m \in \mathbb{R}^D, m = 1, \dots, M$, 以至于我们可以定义函数

$$F(x) = \sum_{m=1}^M v_m \phi(w_m^T x + b_m), \quad (4.37)$$

作为函数 f 的近似实现, 即

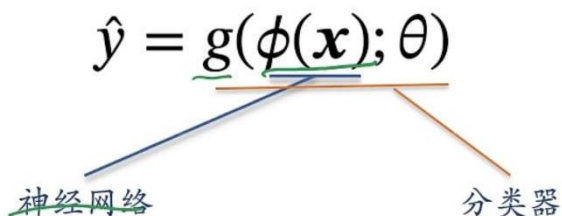
$$|F(x) - f(x)| < \epsilon, \forall x \in \mathcal{J}_D, \quad (4.38)$$

其中 $\epsilon > 0$ 是一个很小的正数.

$$y = \phi(x; w, b)$$

根据通用近似定理，对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络，只要其隐藏层神经元的数量足够，它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。

神经网络可以作为一个万能函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。



比如这个 Φ 就是用神经网络来实现，然后得到表示，之后送到分类器 g 里面得到 y 。

如果 $g(\cdot)$ 为 Logistic 回归，那么 Logistic 回归分类器可以看成神经网络的最后一层。

参数学习

▶ 对于多分类问题

▶ 如果使用 Softmax 回归分类器，相当于网络最后一层设置 C 个神经元，其输出经过 Softmax 函数进行归一化后可以作为每个类的条件概率。

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$$

▶ 采用交叉熵损失函数，对于样本 $(\mathbf{x}, \mathbf{y})$ ，其损失函数为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

▶ 给定训练集为 $D = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$ ，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络，得到网络输出为 $\hat{\mathbf{y}}^{(n)}$ ，其在数据集 D 上的结构化风险函数为：

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|\mathbf{W}\|_F^2$$

然后就可以计算每个样本的这个损失函数取平均，然后再加一个参数 w 的正则化 (带二分之一会方便化简)，相当于对所有的 w 进行一个平方和。

Frobenius 范数

$$\|\mathbf{W}\|_F^2 = \sum_{l=1}^L \sum_{i=1}^{M_l} \sum_{j=1}^{M_{l-1}} \underbrace{(w_{ij}^{(l)})^2}_{\text{计算}} \quad \text{▶ 梯度下降}$$

$$\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{W}^{(l)}}$$

$$\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$

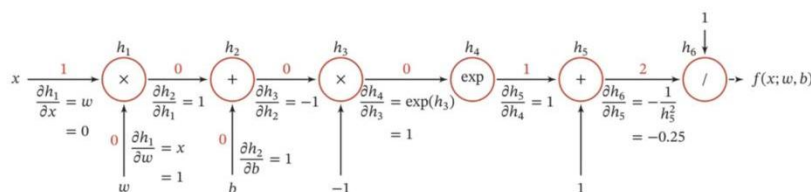
风险函数关于每一层的 w 以及每一层的 b 的梯度，然后就更新就可以了。

计算图与自动微分

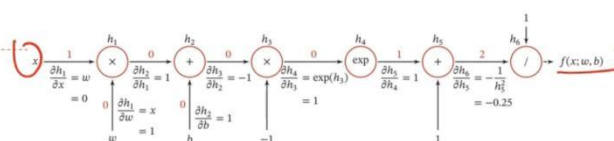
► 自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}$$

► 计算图



计算图



函数	导数
$h_1 = x \times w$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$

$$\frac{\partial f(x; w, b)}{\partial w} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w}$$

$$\frac{\partial f(x; w, b)}{\partial b} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial b}$$

当 $x = 1, w = 0, b = 0$ 时, 可以得到

$$\begin{aligned} \frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\ &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\ &= 0.25. \end{aligned}$$

反向模式和反向传播的计算梯度的方式相同。

如果函数和参数之间有多条路径, 可以将这多条路径上的导数再进行相加, 得到最终的梯度。

静态计算图是在编译时构建计算图, 计算图构建好之后在程序运行时不能改变。(如 tensorflow) 静态计算图在构建时可以进行优化, 并行能力强, 但是灵活性比较差。

动态计算图是在程序运行时动态构建。(如 pytorch) 动态计算图不容易优化, 当不同输入的网络结构不一致时, 难以并行计算, 但是灵活性比较高。

优化问题:

难点：1. 非凸优化问题：即存在局部最优而非全局最优解，影响迭代。

2. 梯度消失问题：下层参数比较难调。

3. 参数过多，影响训练。

卷积的作用：

▶低通滤波/高通滤波

▶滤波器 $w = [1/3, 1/3, 1/3]$ 可以检测信号序列中的低频信息

▶滤波器 $w = [1, -2, 1]$ 可以检测信号序列中的高频信息

卷积类型

▶卷积的结果按输出长度不同可以分为三类：

▶窄卷积：步长 $S = 1$ ，两端不补零 $P = 0$ ，输出长度为 $M - K + 1$

▶宽卷积：步长 $S = 1$ ，两端补零 $P = K - 1$ ，输出长度 $M + K - 1$

▶等宽卷积：步长 $S = 1$ ，两端补零 $P = (K - 1)/2$ ，输出长度 M

目前的文献中，卷积一般默认为等宽卷积。

在图像处理中，图像是以二维矩阵的形式输入到神经网络中，因此需要二维卷积。

一个输入信息 X 和滤波器 W 的二维卷积定义为

$$Y = W * X,$$

$$y_{ij} = \sum_{u=1}^U \sum_{v=1}^V w_{uv} x_{j-u+1, i-v+1}.$$

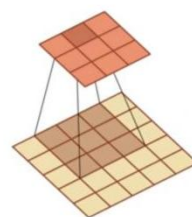
1	1	1	1	1
-1	0	-3	0	1
2	1	1	-1	0
0	-1	1	2	1
1	2	1	1	1

*

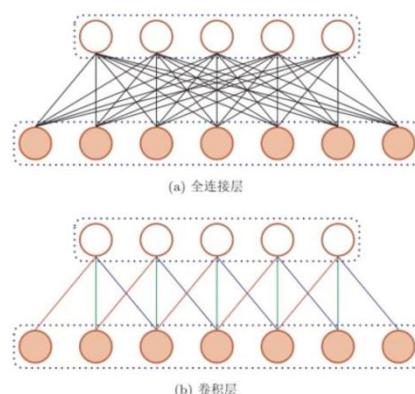
1	0	0
0	0	0
0	0	-1

=

0	-2	-1
2	2	4
-1	0	0



►用卷积层代替全连接层

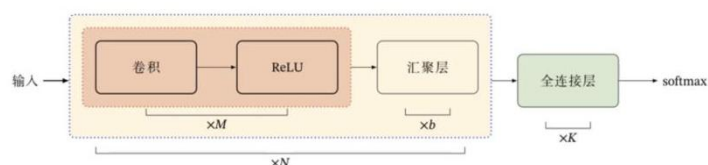


卷积操作的目标：提取特征。

特征映射 (Feature Map)：图像经过卷积后得到的特征。

卷积网络结构

►卷积网络是由卷积层、汇聚层、全连接层交叉堆叠而成。



►一个卷积块为连续M个卷积层和b个汇聚层 (M通常设置为2~5, b为0或1)。一个卷积网络中可以堆叠N个连续的卷积块, 然后在接着K个全连接层 (N的取值区间比较大, 比如1~100或者更大; K一般为0~2)

如何增加输出单元的感受野：增加卷积核的大小；增加层数来实现；在卷积之前进行实行汇聚操作。

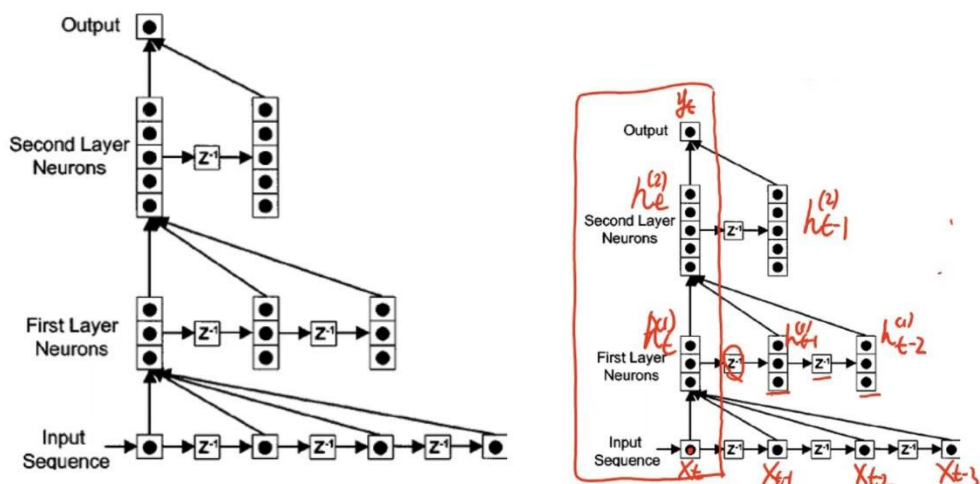
空洞卷积：通过给卷积核插入空洞来变相地增加其大小。

如何给网络增加记忆能力：时延神经网络 (TDNN)。

建立一个额外的延时单元，用来存储网络的历史信息 (可以包括输入、输出、隐状态等)

$$h_t^{(l)} = f(h_t^{(l-1)}, h_{t-1}^{(l-1)}, \dots, h_{t-K}^{(l-1)})$$

这样，前馈网络就具有了短期记忆的能力。



如何给网络增加记忆能力？

▶ 自回归模型 (Autoregressive Model, AR)

▶ 一类时间序列模型，用变量 y_t 的历史信息来预测自己

$$y_t = w_0 + \sum_{k=1}^K w_k y_{t-k} + \epsilon_t$$

▶ $\epsilon_t \sim N(0, \sigma^2)$ 为第 t 个时刻的噪声

▶ 有外部输入的非线性自回归模型 (Nonlinear Autoregressive with Exogenous Inputs Model, NARX)

$$y_t = f(x_t, x_{t-1}, \dots, x_{t-K_x}, y_{t-1}, y_{t-2}, \dots, y_{t-K_y})$$

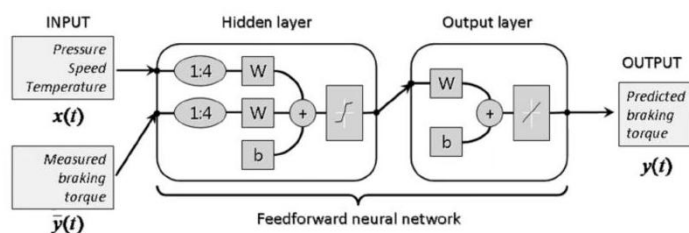
▶ 其中 $f(\cdot)$ 表示非线性函数，可以是一个前馈网络

▶ K_x 和 K_y 为超参数。

第一个公式： w_0 是个可学习的参数，是个偏置。

第二个公式： 外部输入是 x_t 就当前的输入以及前面 K_x 个时刻的历史输入。

非线性自回归模型

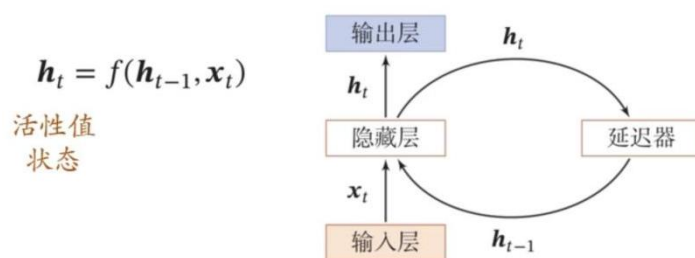


可以看到， $y(t)$ 输出不光和当前时刻的输入 x_t 相关，还

与之前的输出 $\bar{y}$ 相关。中间的 Feedforward neural network 是 $f(\cdot)$ ，也就是一个神经网络。

循环神经网络 (RNN):

循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。



可以通过延时器拿到它的上一个时刻的状态。就是 h_t 等于 $f(h_{t-1})$ 即上一个时刻的状态，还有当前时刻的输入一起更新了这个 h_t ，就当前时刻的状态。

神经网络的优化本身是一个非凸优化问题。

网络非常深的时候不一定要找到全局最优解，因为往往会过拟合。

优化算法：小批量随机梯度下降

几个关键因素：小批量样本数量 k ；梯度；学习率。

批量大小怎么选，也就是每次的梯度的计算都带有一定的随机性，也就是这个方向并不是最优的。

批量大小不影响随机梯度的期望，但会影响随机梯度的方差。

批量越大，随机梯度的方差越小，引入噪声也越小，训

练越稳定，可以设置较大学习率。

批量较小时，需要设置较小的学习率，否则模型会不收敛。

为了提高优化的效率，通常对学习率做一个衰减。比如一开始设置一个初始的学习率，比如走的步长大一点，后面随着这个更新，每次走的步长也就越来越小，这就叫学习率衰减。

为什么要做周期性的学习率调整，就是为了跳出局部最优。比如陷到某个局部最优时，突然增大一下学习率，有助于跳出局部最优。但是也会导致找到的局部最优找到的是一个更平坦的局部最优。假如现在有两个局部最优，一个比较尖锐，另一个比较平坦。那个平坦的突然变下学习率，它基本上还会在这个区域活动，就是它不会走得太远。但如果是一个比较尖锐的局部最优，突然增大一下学习率，就可能跳出这个区域。所以周期性的学习率调整会使得神经网络收敛于更加平坦的局部最优，从而去提高模型。它的鲁棒性也就是不会受某个参数的变化而导致性能出现大幅度的变动。

学习率衰减

▸ 逆时衰减 (Inverse Time Decay)

$$\alpha_t = \alpha_0 \frac{1}{1 + \beta \times t}$$

▸ 指数衰减 (Exponential Decay)

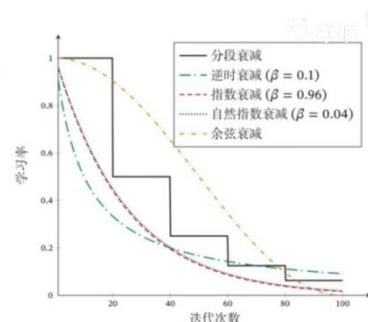
$$\alpha_t = \alpha_0 \beta^t$$

▸ 自然指数衰减 (Natural Exponential Decay)

$$\alpha_t = \alpha_0 \exp(-\beta \times t)$$

▸ 余弦衰减 (Cosine Decay)

$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos\left(\frac{t\pi}{T}\right) \right)$$



梯度截断

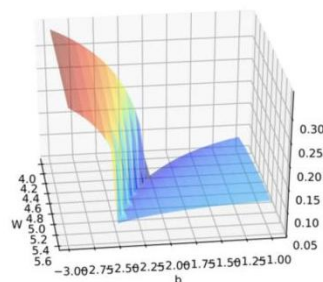
▸ 梯度截断是一种比较简单的启发式方法，把梯度的模限定在一个区间，当梯度的模小于或大于这个区间时就进行截断。

▸ 按值截断

$$\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a)$$

▸ 按模截断

$$\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|} \mathbf{g}_t$$



梯度截断是用来解决梯度爆炸问题的。

类别		优化算法
学习率策略	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam ≈ 动量法 + RMSprop

参数不能初始化为 0；偏置 (Bias) 通常用 0 来初始化。

随机初始化



► Gaussian分布初始化

- Gaussian初始化方法是最简单的初始化方法，参数从一个固定均值（比如0）和固定方差（比如0.01）的Gaussian分布进行随机初始化。

► 均匀分布初始化

- 参数可以在区间 $[-r, r]$ 内采用均匀分布进行初始化。

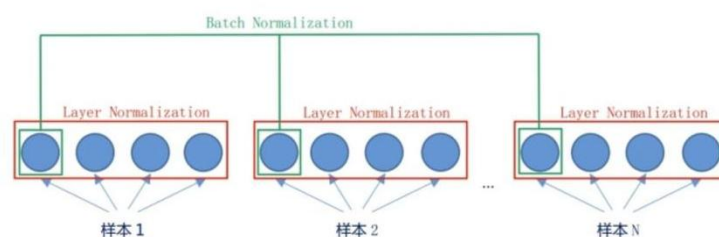
批量规范化

优点：提高优化效率；隐形的正则化方法

解释：在训练时，神经网络对一个样本的预测不仅和该样本自身相关，也和同一批次中的其他样本相关。由于在选取批次时具有随机性，因此使得神经网络不会“过拟合”到某个特定样本，从而提高网络的泛化能力。

缺点：小批量样本的数量不能太小；无法应用到循环神经网络。

批量规范化VS层规范化



超参数优化

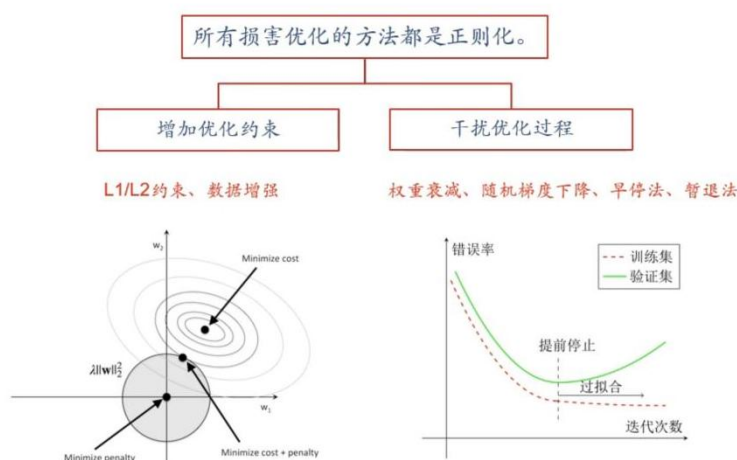
► 网格搜索 (Grid Search)

- 假设总共有K个超参数，第k个超参数的可以取 m_k 个值。
- 如果参数是连续的，可以将参数离散化，选择几个“经验”值。比如学习率 α ，我们可以设置

$$\alpha \in \{0.01, 0.1, 0.5, 1.0\}$$

- 这些超参数可以有 $m_1 \times m_2 \times \cdots \times m_K$ 个取值组合。

正则化 (regularization)



不要收敛到经验风险最小点，否则会导致过拟合。

早停法：使用一个验证集来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。

权重衰减 (Weight Decay)

- 通过限制权重的取值范围来干扰优化过程，降低模型能力
- 在每次参数更新时，引入一个衰减系数 β

$$\theta_t \leftarrow (1 - \beta)\theta_{t-1} - \alpha \mathbf{g}_t$$

一般取值比较小，
比如0.0005

暂退法 (Dropout Method)

▶ 神经层 $y = f(Wx + b)$

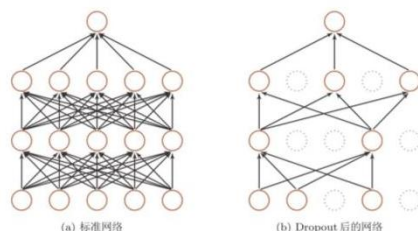
▶ 引入一个掩蔽函数使得

$$y = f(W\text{mask}(x) + b)$$

$$\text{mask}(x) = m \odot x$$

$m \in \{0,1\}^D$ 掩码

通过以概率为 p 的
贝努力分布随机生成



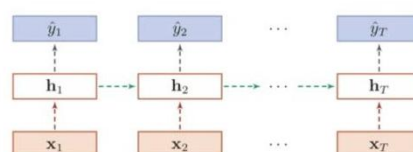
循环神经网络上的暂退：当在循环神经网络上应用暂退法，不能直接对每个时刻的隐状态进行随即暂退，这样会损害循环网络在时间维度上记忆能力。

变分Dropout

▶ 根据贝叶斯学习的解释，暂退法是一种对参数 θ 的采样。

▶ 每次采样的参数需要在每个时刻保持不变。

▶ 在对循环神经网络上使用暂退法时，需要对参数矩阵的每个元素进行随机暂退，并在所有时刻都使用相同的暂退掩码。



相同颜色表示使用相同的暂退掩码

数据增强也可以提高网络泛化能力。

数据增强就是人工构造新的样本来增加训练数据的数量和多样性。

增强方法：翻转、旋转、缩放、平移、加噪声。

标签平滑 (Label Smoothing)



- ▶在输出标签中添加噪声来避免模型过拟合。
- ▶一个样本 x 的标签一般用onehot向量表示

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^T \quad \text{硬目标 (Hard Targets)}$$

- ▶引入一个噪声对标签进行平滑，即假设样本以 ϵ 的概率为其它类。平滑后的标签为

$$\tilde{\mathbf{y}} = [\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1 - \epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}]^T$$

注意力机制是一种有效的从大量的输入信息或者外部信息中筛选出一些和任务相关的特定信息。可以减少一些冗余信息的干扰，大幅的降低模型的计算量。

记忆增强网络是指给神经网络增加一块外部的记忆，使得可以增强网络的记忆能力。

一种是结构化的外部记忆，用一个矩阵或者向量来存储。是一种自下而上的注意力，也就是不需要去主动关注，只要输入信息刺激性足够强，比如鲜艳的图片或者字这些自带一些显著的特征，所以不需要主动去聚焦，它会通过神经系统得到相应的关注。

自上而下的注意力是要带着问题去关注的。

例如：当一个人在吵闹的鸡尾酒会上和朋友聊天时，尽管周围噪音干扰很多，他还是可以听到朋友的谈话内容，而忽略其他人的声音。同时，如果未注意到的背景声中有重要的词(比如他的名字)，他会马上注意到。

就是把注意力主动关注到我和朋友之间的对话，背景音

就相对来讲忽略掉了，但是背景音不是被完全忽略的，如果背景音中有些重要的显著的信息，会立马注意到背景音传过来的这个信号。就是说应该以更高的注意力去关注我们关心的内容，但是不关心的内容也不是完全忽略的，它是以较低的注意力或者较低的权重来关注它。

图灵完备：一种数据操作规则，比如一种计算机编程语言，可以实现图灵机的所有功能，解决所有的可计算问题。

一个完全连接的循环神经网络可以近似解决所有的可计算问题。

所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟。

循环神经网络

优点：引入（短期）记忆；图灵完备。

缺点：长程依赖问题；记忆容量问题；并行能力问题。

注意力机制的变体

▶ 多头注意力 (multi-head attention)

▶ 利用多个查询 $Q = [q_1, \dots, q_M]$ ，同时从输入信息中选取多组信息。每个“注意力头”关注输入信息的不同部分

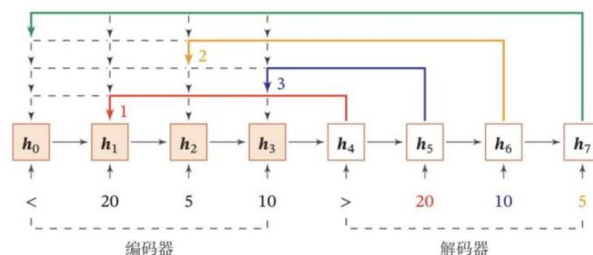
$$\text{att}((K, V), Q) = \text{att}((K, V), q_1) \oplus \dots \oplus \text{att}((K, V), q_M)$$

用多个查询去并行的从输入信息中选取多组信息，比如准备 m 个查询。用 q_1 用 attention 去从输入信息中选一些信息，一直用 q_m 去选，总共选取了 M 个信息，其实就是得到了 M 个向量，然后把 M 个向量拼起来，就得到了最终多头注意力选出来的信息。用多头是希望每个注意力每个头关注输

入信息的不同部分，这样可以选择出更全面的信息。

指针网络 (Pointer Network)

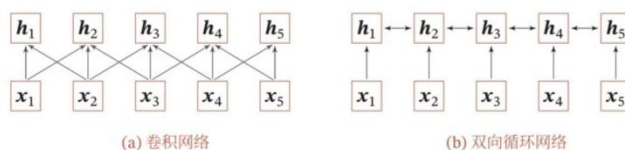
- 只利用注意力机制中的第一步，将注意力分布作为一个软性的指针 (pointer) 来指出相关信息的位置。



指针网络通常用在其他神经网络中作为一个模块，用来指出候选信息中哪些和我们的信息符合。比如说任务查询向量 q 相关的这个信息在哪里就可以了。比如阅读理解，假设答案出现在原文中，只需用一个指针指出来答案在哪里就可以了，不需要把这个答案加权汇总得到它的一个向量表示。

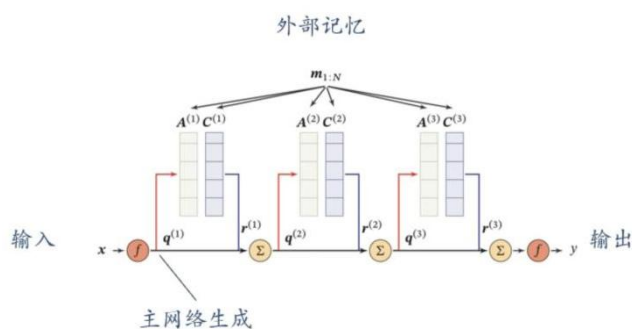
变长序列的建模

- 当使用神经网络来处理一个变长的向量序列时，我们通常可以使用卷积网络或循环网络进行编码来得到一个相同长度的输出向量序列。



比如说 x_1 到 x_5 是一个序列，用卷积来建模时，固定一个卷积核，然后做这个序列上的一维卷积滑动就可以了。比如 h_2 它的表示就来自于 x_1 、 x_2 、 x_3 的组合。

还有一种方式是循环网络。直接建模 h 之间的依赖关系。

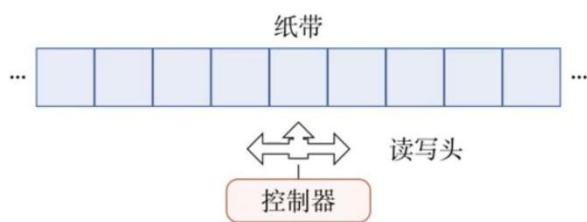


把问题输入到一个主网络里面， f 是一个前馈网络或者是循环网络，比如把 x 输入到 f 中得到一个 $q^{(1)}$ ，这个 q 就是用来做子注意力里面的 query 的，用这个 $q^{(1)}$ 去对 $A^{(1)}$ 里面的每一个向量进行打分，计算一个注意力权重，然后经过一个 softmax 算出注意力权重，再乘上 $C^{(1)}$ ，就相当于对 $C^{(1)}$ 里面的所有向量都进行一个加权组合，然后得到了 $r^{(1)}$ ， $r^{(1)}$ 就是读出来的内容。然后把 $r^{(1)}$ 直接加上一个 $q^{(1)}$ 就得到了下一步的 $q^{(2)}$ 。然后 $r^{(3)} + q^{(3)}$ 输入到 f 里面，最终输出 y 预测答案。

$$\begin{aligned} \underline{q^{(1)}} &\leftarrow f(x) \\ \underline{r^{(1)}} &= C^{(1)} \text{softmax}(A^{(1)T} q^{(1)}) \\ \underline{q^{(2)}} &\leftarrow r^{(1)} + q^{(1)} \end{aligned}$$

story 作为外部记忆。 m_1 到 N 表示整个 story 的长度，其中每个 m 就是 story 里面的每个单词。

图灵机是一种抽象数学模型，可以用来模拟任何可计算问题。



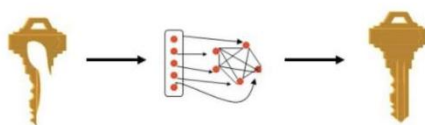
控制器负责接收外面的输入然后产生输出。

纸带可以用来记录一些信息。所以可以看成是一个无限长的记忆。所以基本上认为它可以解决任何的可计算问题。

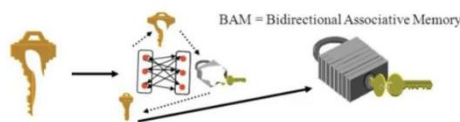
控制器通过读写头和纸带进行交互。

联想记忆

▶ 自联想



▶ 异联想



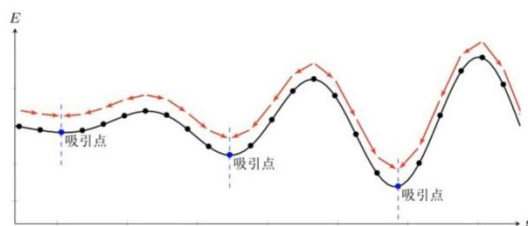
联想记忆：想要读出记忆中的内容时，通过的是联想的方式，也就是说有一个相似或相关的输入，然后来检索它，而不是通过地址的方式去检索它。

自联想：比如说想检索出一个钥匙，那么输入就是一个残缺不全的或者加了噪声的钥匙，然后进入到联想记忆的一个网络中，然后把完整的钥匙给检索出来。

异联想：输入输出不是同一个物体，比如给一个钥匙，然后输入网络中让检索出一把锁。就是它只要相关，就可以联想出来。

检索过程（联想记忆）

▶ 给定一个外部输入，网络经过演化，会达到某个稳定状态。



检索过程

- ▶ 每个吸引点 u 都对应一个“管辖”区域。如果输入向量 x 落入这个区域，网络最终会收敛到 u 。
- ▶ 吸引点可以看作是网络中存储的信息。

存储过程（学习过程）

▶ 信息存储是指将一组向量 $x^{(1)}, \dots, x^{(N)}$ 存储在网络中的过程。

- ▶ 存储过程主要是调整神经元之间的连接权重
- ▶ 神经元 i 和 j 之间的连接权重

$$w_{ij} = \frac{1}{N} \sum_{n=1}^N x_i^{(n)} x_j^{(n)}$$

x_i, x_j 都是正负 1。

如果两个神经元经常同时激活，则它们之间的连接加强；

如果经常不同时激活，则连接消失。

存储容量：每 1000 个节点可以存储 138 组向量。

联想记忆具有存储和检索的功能，可以利用其来增加网络容量。

监督学习：建立映射关系 $f: x \rightarrow y$ 。

无监督学习：从无标签的数据中学习出一些有用的模式。

聚类：将样本集合中相似的样本分配到相同的类，不相似的样本分配到不同的类，使得类内样本间距较小而类间样本间距较大。

理解不同，聚类的结果也不同。

在不同的分类任务中，核心原则就是怎样去定义两个点之间相似度或者间距。

核心概念

▸ 类内间距

▸ 样本间平均距离 $avg(C) = \frac{2}{|C|(|C|-1)} \sum_{1 \leq i < j \leq |C|} d_{ij}$

▸ 样本间最大距离(直径, diameter)

$$dia(C) = \max_{1 \leq i < j \leq |C|} d_{ij}$$

▸ 类间距离(设为 C_p, C_q)

▸ 样本间最短距离

$$D_{pq} = \min\{d_{ij} | x^{(i)} \in C_p, x^{(j)} \in C_q\}$$

▸ 样本均值间距离

$$D_{pq} = d_{\mu_p \mu_q}$$

聚类效果评估

▸ 外部指标 (external index)

▸ 有外部参考聚类结果

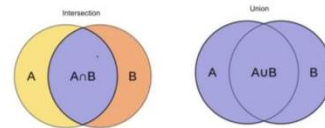
▸ Jaccard系数 (Jaccard Coefficient)

▸ FM指数 (Fowlkes and Mallows Index)

▸ Rand指数 (Rand Index)

混淆矩阵		预测结果	
		同类	不同类
真实结果	同类	#TP	#FN
	不同类	#FP	#TN

数据集中每组数据($x^{(i)}, x^{(j)}$)
在预测/参考中结果



$$JC = \frac{\#TP}{\#TP + \#FN + \#FP}$$

Jaccard系数

外部评价：给定一堆 x ，其实知道其标签 y 但不告诉你，聚类完以后，用 y 去衡量距离算法的好坏。

内部指标：

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{avg(C_i) + avg(C_j)}{d_{\mu_i \mu_j}} \right)$$

类内间距
类内间距

类间间距

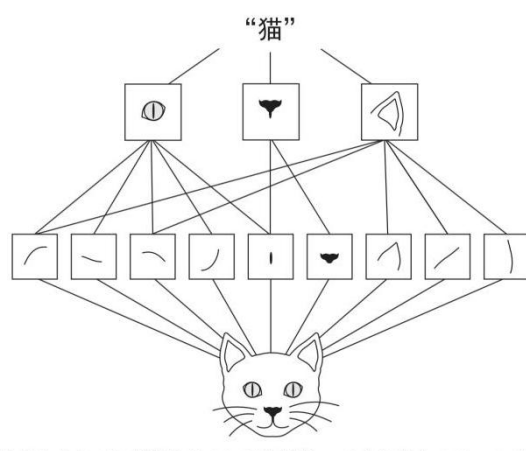
DBI 指数如果越大，它的聚类结果就越不好。

回调函数 `EarlyStopping` 和 `ModelCheckpoint`:

训练模型时，很多事情一开始无法预测，尤其是你无法预测需要多少轮才能达到最佳验证损失。一种好的处理方法是，发现验证损失不再改善时，停止训练。这可以通过 `EarlyStopping` 回调函数来实现。如果监控的目标指标在设定的轮数内不再改善，那么可以用 `EarlyStopping` 回调函数中断训练。比如，这个回调函数可以在刚开始过拟合时就立即中断训练，从而避免用更少的轮数重新训练模型。这个回调函数通常与 `ModelCheckpoint` 结合使用，后者可以在训练过程中不断保存模型（你也可以选择只保存当前最佳模型，即每轮结束后具有最佳性能的模式）。

利用 `tf.function` 加快运行速度。可以为评估步骤函数添加 `@tf.function` 装饰器。

卷积：图像可以被分解为局部模式，比如边缘、纹理等。



目标检测

前提是对分类网络有了解。

One-Stage(单阶段): 它直接在单个网络中同时预测物体的类别和位置, 而不需要区域建议 (Region Proposal) 的生成和后处理步骤。这种方法通常比 Two-Stage (双阶段) 目标检测方法更快, 因为它们避免了生成和评估多个候选区域的需要。One-stage 检测器通常用于需要实时或近实时目标检测的应用中。

One-stage 目标检测器的关键特点:

速度: 由于省略了区域建议生成和后续的分类步骤, one-stage 检测器通常可以更快地处理图像。

简化的流程: 整个检测过程在一个网络中完成, 简化了训练和推理流程。

直接预测: 网络直接在特征图上预测边界框和类别概率, 无需额外的后处理步骤。

适用性: 适用于需要快速处理的场景, 如视频监控、自动驾驶等。

常见的 One-stage 目标检测模型:

YOLO:

YOLO 将目标检测视为一个回归问题, 直接在图像中预测边界框和类别概率。

SSD:

SSD 在不同尺度的特征图上预测边界框，这使得它能够检测不同大小的物体。

SSD 使用默认框 (Default Boxes) 来预测边界框，这些框在训练前已经定义好。

FCOS:

FCOS 是一种完全卷积的 one-stage 目标检测器，它不需要锚点 (anchor) 或网格单元，直接预测物体的边界框。

FCOS 通过预测边界框的中心点和边界框与中心点的距离来确定边界框的位置。

One-stage 目标检测器的挑战:

类别不平衡: 在训练过程中，背景样本通常远多于前景样本，这可能导致模型对背景的预测过于自信。

精度: 尽管 one-stage 检测器在速度上有优势，但它们通常在精度上不如 two-stage 检测器，尤其是在检测小物体时。

多尺度检测: 处理不同尺度的物体是一个挑战，需要模型能够适应不同大小的特征图。

总的来说，one-stage 目标检测器因其速度快、易于实现而受到广泛关注，尤其适用于需要实时处理的场景。然而，它们在处理复杂场景和检测小物体时可能面临挑战。随着研究的进展，one-stage 检测器的精度和鲁棒性也在不断提高。

Two-Stage(双阶段): 它将目标检测任务分为两个阶段进行: 区域建议 (Region Proposal) 生成和区域分类与边界框回归。这种检测方法通常能够提供更高的精度, 但相应的计算成本和推理时间也更长。Two-stage 检测器广泛应用于精度要求较高的场景。

第一阶段: 生成区域建议 (Region Proposals)

在第一阶段, 目标是快速生成一系列可能包含目标物体的区域建议。这些区域建议通常是通过一种称为区域建议网络 (Region Proposal Network, RPN) 的网络结构来生成的。RPN 通常是一个滑动窗口网络, 它在特征图的不同位置生成多个候选区域。

1. 特征提取: 首先, 使用卷积神经网络 (CNN) 从输入图像中提取特征图。

2. 区域建议网络 (RPN): RPN 在特征图上滑动, 为每个位置生成多个不同尺度和比例的锚点 (anchors)。然后, RPN 为每个锚点预测一个分数, 表示该锚点包含物体的可能性。

3. 候选区域生成: 根据 RPN 预测的分数, 选择分数较高的锚点作为候选区域。这些候选区域可能会经过非极大值抑制 (NMS) 来去除重叠的区域, 从而保留最有可能包含目标物体的区域。

第二阶段: 分类与边界框回归

在第二阶段, 对第一阶段生成的区域建议进行进一步处

理，包括分类和边界框回归。

1. ROI 池化：将第一阶段生成的区域建议映射回原始特征图，然后通过 ROI 池化层将这些不同大小的区域建议转换为固定大小的特征向量。

2. 分类：使用一个全连接层对 ROI 池化层输出的特征向量进行分类，以确定区域建议中包含的物体类别。

3. 边界框回归：同时，另一个全连接层对特征向量进行边界框回归，以精确调整区域建议的位置和大小，使其更准确地包围目标物体。

4. 后处理：最后，对分类和边界框回归的结果进行后处理，包括应用阈值来去除低置信度的预测，以及再次使用非极大值抑制来合并重叠的预测。

Two-stage 目标检测器的关键特点：

高精度：由于引入了区域建议和后处理步骤，two-stage 检测器通常能够提供更高的检测精度。

区域建议：第一阶段生成一系列可能包含目标的区域建议，这些建议随后在第二阶段进行分类和边界框回归。

后处理：包括非极大值抑制等步骤，用于合并重叠的边界框并选择最终的检测结果。

适用性：适用于对精度要求较高的应用，如医学图像分析、精细物体检测等。

常见的 Two-stage 目标检测模型：

R-CNN:

R-CNN 首先使用选择性搜索生成区域建议，然后在这些建议上应用 CNN 进行分类和边界框回归。

R-CNN 的后续版本，如 Fast R-CNN 和 Faster R-CNN，通过引入区域建议网络（Region Proposal Network, RPN）和改进的训练策略，提高了检测速度和精度。

Fast R-CNN:

Fast R-CNN 改进了 R-CNN 的训练过程，通过在单个网络中共享卷积特征来提高效率。

它使用 ROI 池化层来处理不同大小的区域建议。

Faster R-CNN:

Faster R-CNN 引入了 RPN 来生成区域建议，从而实现了端到端的训练。

RPN 与检测网络共享卷积特征，从而提高了检测速度。

Two-stage 目标检测器的挑战:

速度：由于需要生成区域建议和进行后处理，two-stage 检测器通常比 one-stage 检测器慢。

复杂性：two-stage 检测器的训练和推理过程更复杂，需要更多的计算资源。

类别不平衡：在生成区域建议时，可能会产生大量的负样本（不包含目标的区域），这可能导致类别不平衡问题。

总的来说，two-stage 目标检测器因其高精度而受到广

泛关注，尤其适用于对检测精度要求较高的场景。然而，它们在速度和计算资源方面可能面临挑战。